

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Verification of binarised neural
networks using ASP**

Bachelor's Thesis

JINDŘICH MATUŠKA

Brno, Spring 2024

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Verification of binarised neural
networks using ASP**

Bachelor's Thesis

JINDŘICH MATUŠKA

Advisor: RNDr. Samuel Pastva, PhD.

Department of Computer Systems and Communications

Brno, Spring 2024

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Jindřich Matuška

Advisor: RNDr. Samuel Pastva, PhD.

Acknowledgements

My thanks go to my family, which has provided me with a firm ground for my whole life, to my friends whom I could spend my free time and who were distracting me from the never-ending study duties, to all my teachers, who had patience with me and helped me grow, and finally to my advisor, who has provided me with this assignment and was my support when writing this thesis.

Abstract

Deep neural networks are state-of-the-art technology. Using them in critical real-life applications carries a risk of failure. For this, verification of their properties is needed. This thesis explores the possibility for the use of answer set programming paradigm in this task. It implements a quantitative verifier for binarized neural networks, a special case of deep neural networks, using this paradigm. It also demonstrates the use of this verifier on a network trained on the MNIST dataset.

Keywords

verification, binarized neural networks, answer set programming, Clingo, robustness

Contents

Introduction	3
1 Binarised neural networks	5
1.1 Definition of Binarised neural network	5
1.1.1 Deep neural network	5
1.1.2 Binarised neural network	8
1.2 Robustness of Binarised neural network	13
1.2.1 Definition of robustness	13
1.2.2 Definition of input regions	18
2 Answer set programming	20
2.1 Extended logic program	20
2.1.1 Syntax of Extended logic program	21
2.1.2 Semantics of extended logic program	23
2.2 Clingo	30
2.2.1 Clingo language	30
2.2.2 Gringo	30
2.2.3 Clasp	30
3 ASP encoding of BNN robustness	32
3.1 Analysis of BNN	32
3.2 ASP encoding of BNN	34
3.2.1 Encoding of a perceptron	37
3.2.2 Encoding of an Argmax layer	39
3.2.3 Encoding of input regions	43
3.2.4 Encoding of robustness	44
3.3 Encoding robustness of BNN into logic program	45
3.3.1 Structure of saved BNN	45
3.3.2 Transformation of BNN	47
3.3.3 Evaluator framework	49
4 Evaluation	50
4.1 Methodology of evaluation	50
4.1.1 Consistency of evaluation	50
4.2 Evaluation of encodings	52

4.2.1	Evaluation of perceptron computation	52
4.2.2	Evaluation of argmax computation	54
4.2.3	Evaluation of fixed bits encoding	58
4.3	Evaluation of performance of the implemented frame- work	58
4.3.1	Over hamming distance	59
4.3.2	Over fixed bits	59
Conclusion		62
A Proofs and examples		63
A.1	Proofs	63
A.2	Examples	65
B Clingo		68
B.1	Clingo language	68
B.1.1	Basic syntax of Clingo language	68
B.1.2	Extensions of ASP in Clingo language	72
B.2	Gringo	79
B.2.1	Aspif format	80
B.2.2	Rule statements	80
B.2.3	Show statements	85
B.3	Clasp	86
C Code of BNN verifier		87
D Specifications of system used for evaluation		88
D.1	Hardware	88
D.2	Software	88
E Evaluation of models		89
Bibliography		91

List of Tables

1.1	Used notation	5
3.1	Semantics of encoding BNN into Clingo-readable file . . .	35
4.1	Architectures of models	50
4.2	Computation time for differnt implementations of percep- tron and input region based on hamming distance	53
4.3	Computation time for differnt implementations of percep- tron and input region based on fixed bits	54
4.4	Computation time for differnt implementations of argmax layer and input region based on hamming distance	55
4.5	Computation time for differnt implementations of argmax layer and input region based on fixed bits	57
4.6	Computation time for differnt implementations of input region based on fixed bits.	58
E.1	Computation time of verification and robustness of all models based on different hamming distance	89
E.2	Computation time of verification and robustness of all models based on different fixed bits	90

List of Definitions

1.1.1	Definition (Perceptron)	6
1.1.2	Definition (Single-layer perceptron)	7
1.1.3	Definition (Multi-layer perceptron)	7
1.1.4	Definition (Binarised perceptron)	9
1.1.5	Definition (Binarised perceptron with batch normalization)	10
1.1.6	Definition (Binarised single-layer perceptron)	12
1.1.7	Definition (Argmax layer)	12
1.1.8	Definition (Binarised multi-layer perceptron)	12
1.2.1	Definition (Quantitative robustness of function)	13
1.2.2	Definition (Qualitative robustness of function F on input region I)	15
1.2.3	Definition (Strict qualitative robustness of function F on input region I)	16
1.2.4	Definition (Input region based on the Hamming distance)	18
1.2.5	Definition (Input region based on fixed bits)	19
2.1.1	Definition (Signature [22])	21
2.1.2	Definition (Terms [22])	21
2.1.3	Definition (Atoms, literals [22])	21
2.1.4	Definition (Rule [19])	22
2.1.5	Definition (Logic program [19])	23
2.1.6	Definition (Ground program [19])	24
2.1.7	Definition (Partial interpretation of a signature)	24
2.1.8	Definition (Consistent partial interpretation)	25
2.1.9	Definition (Basic logic program)	25
2.1.10	Definition (Answer set of a basic logic program [19])	25
2.1.11	Definition (Reduct of a logic program [19])	28
2.1.12	Definition (Answer set of general grounded logic program [19])	28

Thesis assignment

ASP (Answer Set Programming) is a form of constraint programming designed for solving various NP-complete search problems. It is often used as an alternative to SAT/SMT solvers or symbolic algorithms based on BDDs (Binary Decision Diagrams). Meanwhile, SAT/SMT and BDDs are one of the key tools in current verification and validation workflows for binarised neural networks (BNNs) [1]. Conceptually, the goal of this thesis is therefore to explore the possibilities of applying ASP to reason about the behaviour of binarised neural networks. This should supplement the existing SAT/SMT/BDD-based approaches.

More concretely, the student should familiarise themselves with the problem of BNN robustness and the ASP method in general. They should then formulate an ASP encoding of the BNN robustness problem such that the robustness of a network can be validated using a suitable ASP solver (e.g. clingo [2]). The student should then create a prototype implementation of this encoding that will be tested on a collection of reasonable examples (such as the ones from [1]).

Introduction

Deep neural networks (DNN) are state-of-the-art technology. They are used in many real-world applications e.g. medicine, self-driving cars, autonomous systems, many of which are critical. Deep neural networks used in natural language processing have up to hundreds of billions of parameters [3]. That is too many for people to comprehend. Tools for the automation of DNN verification are needed.

While DNNs generally base their computation on floating point numbers, another branch of DNN development, quantized neural networks use for their computation low-bit-width values. This can lower both the size of neural network and power consumption while increasing speed of the network [4, 5]. An extreme example of quantization are Binarised neural networks (BNN) [6], which use binary parameters for their computations, resulting in much cheaper computational price, both when learning and in production, while maintaining near state-of-the-art results [7].

Verification of neural networks is split into two categories. First, qualitative verification, searches through input space looking for any adversarial input, that is an input which results in a wrong output. Second, quantitative verification, searches through input space determining the portion of the input space giving adversarial outputs. While both qualitative and quantitative verifiers for BNNs do exist, they scale only up to millions of parameters for qualitative verification [8] and tens of thousands of parameters for quantitative verification [9].

In this thesis, I provide definition of binarised neural network and the problems of both qualitative and quantitative robustness over arbitrary function. Further I provide a definition of an input region based on hamming distance from base input and based on fixed bits.

The third chapter focusses on Answer set programming (ASP) [10, 11]. In it I provide a definition of a logic program and semantics of ASP using the reduct of a program [12]. In appendix, I also show the syntax and semantics of an integrated ASP solver Clingo [13].

I then implement a framework for quantitative robustness verification of BNNs using Clingo. This implementation allows for verification of input regions based on hamming distance and fixed bits as in [1].

LIST OF DEFINITIONS

As far as I am concerned, this is the first such implementation using the ASP paradigm.

Finally, I evaluate the speed of this verifier on BNN models of various architectures based of PyTorch deep learning platform provided by NPAQ [14] trained in [9] on MNIST [15] dataset.

1 Binarised neural networks

Binary neural networks are a type of Deep neural networks where instead of floating point numbers, binary values are used as inputs and outputs of layers. This reduction of available values can lead to high reduction in the computation time and energy consumption while achieving near state-of-the-art results [16].

Notation

Symbol	Definition
$\mathbb{R}$	set of real numbers
$\mathbb{Z}$	set of whole numbers
$\mathbb{B}$	set of ± 1 -binarised values, $\mathbb{B} = \{-1, 1\}$
K^m	set of vectors on K of length m

Table 1.1: Used notation

1.1 Definition of Binarised neural network

1.1.1 Deep neural network

General neural network is a multilayer perceptron. It consists of perceptrons (neurons) in layers. Layers are further split into the input layer, which does not contain any perceptrons, the output layer and possibly multiple hidden inner layers. Using this structure, general neural network implements a function $\mathbb{R}^m \rightarrow \mathbb{R}^n$, such that m entities in the input layer hold input values, and n perceptrons of the output layer create output values. Input layer does not consist of perceptrons, it only holds input values. Each perceptron computes its inner potential ξ based on the outputs of the previous layer, its output is then determined by an activation function σ .

In case of dense deep neural networks which are discussed in this thesis, the inner potential ξ is computed as a weighted sum of outputs

of all entities (input values or perceptrons outputs) of the previous layer. Many different activation functions σ are used in practice, such as *unit step function*, *logistic sigmoid*, *hyperbolic tangens* or *ReLU*.

Multiple of these perceptrons are then assorted into layers. The input layer does not consist of perceptrons but is directly composed of inputs. The output layer is often represented by perceptrons with activation function different from other layers, such as *argmax* for single-choice classification and *softmax* for classification using probability.

Further follows a more formal definition of a deep neural network corresponding to the one from [17].

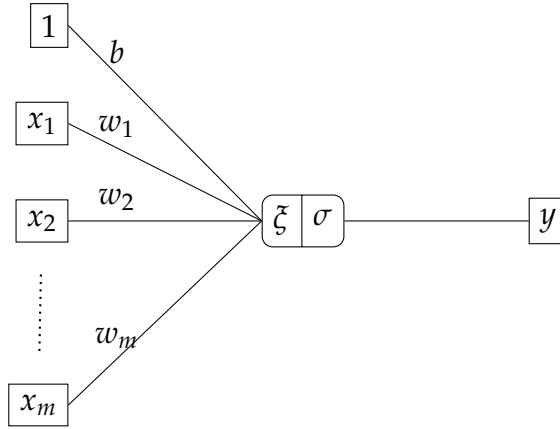


Figure 1.1: Schema of a perceptron

Definition 1.1.1 (Perceptron). Perceptron p is a function from vector of m real numbers to a real number. The function p is a composition of an inner potential ξ and an activation function σ . The inner potential ξ is a weighted sum parametrized by static vector of real numbers $\vec{w}$ of size m and real bias b . The activation function σ can be instantiated by any real-valued function.

To describe a perceptron, it is common to use a schema such as on Figure 1.1.

$$\begin{aligned}
 p &: \mathbb{R}^m \rightarrow \mathbb{R} \\
 \zeta &: \mathbb{R}^m \rightarrow \mathbb{R}, \sigma: \mathbb{R} \rightarrow \mathbb{R} \\
 \zeta(\vec{x}) &= b + \sum_{i=1}^m w_i \cdot x_i \\
 p &= \sigma \circ \zeta
 \end{aligned}$$

Definition 1.1.2 (Single-layer perceptron). Single-layer perceptron t is a function from vector of m real numbers to vector of n real numbers where each value in the resulted vector is computed by a single perceptron.

$$\begin{aligned}
 t &: \mathbb{R}^m \rightarrow \mathbb{R}^n \\
 t(\vec{x}) &= (p_1(\vec{x}), p_2(\vec{x}), \dots, p_n(\vec{x}))
 \end{aligned}$$

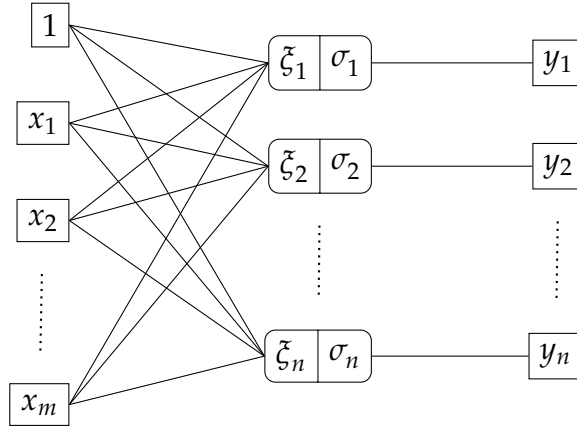


Figure 1.2: Schema of a single-layer perceptron

Definition 1.1.3 (Multi-layer perceptron). Multi-layer perceptron (also called deep neural network) is a convolution of Single-layer perceptrons. The last applied layer t_{d+1} is the called output layer, all other layers are

called hidden layers. The input of a multi-layer perceptron is called input layer, it does not consist of neurons as other layers, only holds the input vector.

A multi-layer perceptron is often symbolised as a graph of nodes composed into layers such as on Figure 1.3. Commonly, values 1 are removed from the schema.

$$\mathcal{N} : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_{d+1}}$$

$$\mathcal{N} = t_{d+1} \circ t_d \circ \dots \circ t_1$$

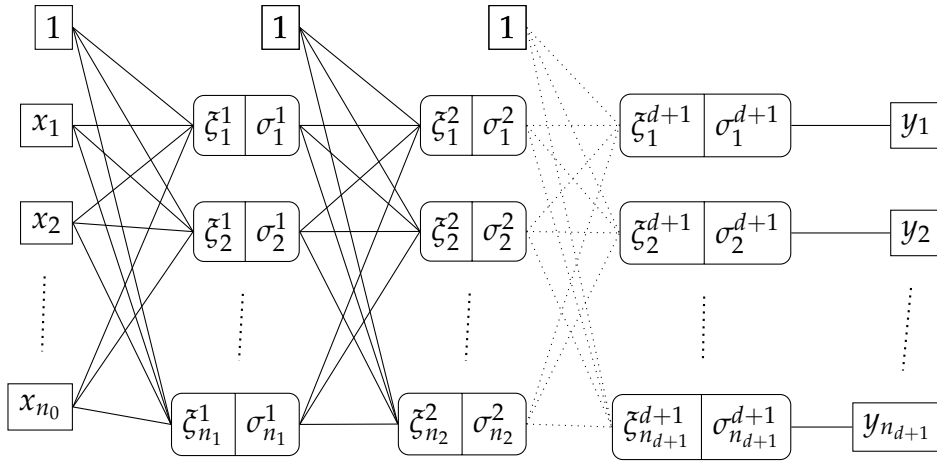


Figure 1.3: Schema of a multi-layer perceptron

1.1.2 Binarised neural network

The computation of general multi-layer perceptron depends on slow multiplication of floating-point numbers. This motivated the idea of a binarized perceptron [18]. Such perceptron constrains both the input space $\vec{x}$ and vector of weights $\vec{w}$ to vectors of binary values -1 and 1 .

The activation function is usually a heavyside step function H .

$$H(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

Computation of this constrained perceptron is faster than of a general perceptron as multiplication of two ± 1 -binarized values can be done with a single *XNOR* gate [18].

From binarized perceptrons, multi-layer perceptron can be built similarly to the general perceptron. In case of the single-choice classification problem, the output layer consists of weighted sums and *argmax* operator to classify the largest sum as the output of the multi-layer perceptron. Such layer is often called after its operator the *argmax* layer.

Further I provide a formal definition of binarised neural network using two definition of binarised perceptron. The first one is useful for the verification task, while the other (binarised perceptron with batch normalization [1]) is commonly used for training of deep neural networks. I show that every binarised perceptron with batch normalization can be transformed into binarised perceptron without batch normalization vice versa. This shows the equivalence of this definition to that of [1].

Definition 1.1.4 (Binarised perceptron). Binarised perceptron $p^{\mathbb{B}}$ is a function from vector of m ± 1 -binarised values to a single ± 1 -binarised value. The function $p^{\mathbb{B}}$ is a composition of an inner potential ζ and a heavyside step function H . The inner potential ζ is weighted sum parametrized by static vector of ± 1 -binarised values $\vec{w}$ of size m , and real bias b .

$$\begin{aligned} p^{\mathbb{B}} &: \mathbb{B}^m \rightarrow \mathbb{B} \\ \zeta &: \mathbb{B}^m \rightarrow \mathbb{R}, H : \mathbb{R} \rightarrow \mathbb{B} \\ \zeta(\vec{x}) &= b + \sum_{i=1}^m w_i \cdot x_i \\ p^{\mathbb{B}} &= H \circ \zeta \end{aligned}$$

Definition 1.1.5 (Binarised perceptron with batch normalization). Binarised perceptron with batch normalization $\hat{p}^{\mathbb{B}}$ is a function from vector of m ± 1 -binarised values to a single ± 1 -binarised value. The function $\hat{p}^{\mathbb{B}}$ is a composition of an inner potential ξ , batch normalization function ρ and a heavyside step function H . The inner potential ξ is weighted sum parametrized by static vector of ± 1 -binarised values $\vec{w}$ of size m , and real bias b . The batch normalization function ρ is a function on real numbers parametrized by real values $\alpha, \gamma, \mu, \sigma$.

$$\begin{aligned}\hat{p}^{\mathbb{B}} &: \mathbb{B}^m \rightarrow \mathbb{B} \\ \xi &: \mathbb{B}^m \rightarrow \mathbb{R}, \rho : \mathbb{R} \rightarrow \mathbb{R}, H : \mathbb{R} \rightarrow \mathbb{B} \\ \xi(\vec{x}) &= b + \sum_{i=1}^m w_i \cdot x_i \\ \rho(x) &= \alpha \cdot \left(\frac{x - \mu}{\sigma} \right) + \gamma \\ \hat{p}^{\mathbb{B}} &= H \circ \rho \circ \xi\end{aligned}$$

Lemma 1.1.1. For every Binarised perceptron $p^{\mathbb{B}}$ there is equivalent Binarised perceptron with batch normalization $\hat{p}^{\mathbb{B}}$.

Proof. If the parameters of batch normalization function ρ are set to be $\alpha = \sigma = 1, \gamma = \mu = 0$, function ρ is identity function. With parameters of inner potential ξ unchanged, the following holds

$$p^{\mathbb{B}} = H \circ \xi = H \circ \text{id} \circ \xi = H \circ \rho \circ \xi = \hat{p}^{\mathbb{B}}$$

□

Lemma 1.1.2. For every Binarised perceptron with batch normalization $\hat{p}^{\mathbb{B}}$ there is equivalent Binarised perceptron $p^{\mathbb{B}}$.

Proof. The idea behind this construction comes from [1].

The value of $\hat{p}^{\mathbb{B}}(\vec{x})$ is only determined by the sign of expression $(\rho \circ \xi)(\vec{x})$. Lets thus analyse the inequality $(\rho \circ \xi)(\vec{x}) \geq 0$.

$$(\rho \circ \xi)(\vec{x}) = \alpha \cdot \left(\frac{b + \sum_{i=1}^k w_i \cdot x_i - \mu}{\sigma} \right) + \gamma$$

If $\alpha = 0$, then the expression $\rho \circ \xi$ is a constant function. In that case the perceptron is equivalent to the one with the value of its bias higher than the length of input vector for positive constant perceptron or with negative bias with its absolute value lower than the length of input vector for negative constant perceptron.

By relaxation of the rules for binarised perceptron to allow for value zero in weights, both of these constant perceptrons may be replaced by a perceptron with zero-valued weights and nonnegative resp. negative bias for positive and negative case.

If $\alpha \neq 0$, the expression can be divided by the term $\frac{\alpha}{\sigma}$. In the case of this term being negative, the inequality switches and has to be corrected by further multiplying by -1 .

$$(\rho \circ \xi)(\vec{x}) \cdot \frac{\sigma}{\alpha} = b + \sum_{i=1}^m w_i \cdot x_i - \mu + \frac{\sigma \cdot \gamma}{\alpha}$$

$$(\rho \circ \xi)(\vec{x}) \cdot \frac{\sigma}{\alpha} = (b - \mu + \frac{\sigma \cdot \gamma}{\alpha}) + \sum_{i=1}^m w_i \cdot x_i$$

$$\frac{\alpha}{\sigma} > 0 : (\rho \circ \xi)(\vec{x}) \geq 0 \iff (b - \mu + \frac{\sigma \cdot \gamma}{\alpha}) + \sum_{i=1}^m w_i \cdot x_i \geq 0$$

$$b' = (b - \mu + \frac{\sigma \cdot \gamma}{\alpha}), \vec{w}' = \vec{w}$$

$$\frac{\alpha}{\sigma} < 0 : (\rho \circ \xi)(\vec{x}) \geq 0 \iff (b - \mu + \frac{\sigma \cdot \gamma}{\alpha}) + \sum_{i=1}^m w_i \cdot x_i \leq 0$$

$$\iff (-b + \mu - \frac{\sigma \cdot \gamma}{\alpha}) + \sum_{i=1}^m -w_i \cdot x_i \geq 0$$

$$b' = (-b + \mu - \frac{\sigma \cdot \gamma}{\alpha}), \vec{w}' = -\vec{w}$$

As can be seen, both cases of $\frac{\alpha}{\sigma}$ being positive or negative result in new real bias value b' and vector of weights $\vec{w}'$. These values may then be used as a bias and weights of a binarized perceptron.

$$(\rho \circ \xi)(\vec{x}) = \xi'(\vec{x}) = b' + \sum_{i=1}^m w'_i \cdot x_i$$

□

Remark. The proof of Lemma 1.1.2 is constructive and is used for encoding of the quantitative verification problem as ASP problem.

Remark. As $\sum_{i=1}^m w_i \cdot x_i$ is always a whole number, bottom whole part of bias $\lfloor b \rfloor$ can be used in place of bias in inner layers of BNN. In the output layer however the fractional part can make difference when choosing the maximal input.

Definition 1.1.6 (Binarised single-layer perceptron). Binarised single-layer perceptron is a function $t^{\mathbb{B}}$ from vector of m ± 1 -binarised numbers to vector of n ± 1 -binarised numbers where each value in the result vector is computed by a single perceptron.

$$t^{\mathbb{B}} : \mathbb{B}^m \rightarrow \mathbb{B}^n$$

$$t^{\mathbb{B}}(\vec{x}) = (p_1^{\mathbb{B}}(\vec{x}), p_2^{\mathbb{B}}(\vec{x}), \dots, p_n^{\mathbb{B}}(\vec{x}))$$

Definition 1.1.7 (Argmax layer). Argmax layer is a function t^{am} which returns the mask of maximal value after the weighted sum. This mask has form of a one-hot vector, where only the first position with the maximal value after the weighted sum is assigned value 1, all the other positions are assigned value 0.

$$t^{am} : \mathbb{B}^m \rightarrow \{0, 1\}^n$$

$$t^{am}(\vec{x}) = y, y_k = 1 \iff k = \arg \max_{i=1}^n (\xi_i(\vec{x}))$$

Definition 1.1.8 (Binarised multi-layer perceptron). Binarised multi-layer perceptron (also called binarised neural network) is a convolution of binarised single-layer perceptrons. The last applied layer t^{am} is called the output layer and takes form of argmax layer, all other layers are called hidden (or inner) layers. The input of a binarised multi-layer perceptron is called the input layer.

$$\mathcal{N}^{\mathbb{B}} : \mathbb{B}^{n_0} \rightarrow \{0, 1\}^{n_{d+1}}$$

$$\mathcal{N}^{\mathbb{B}} = t^{am} \circ t_d^{\mathbb{B}} \circ \dots \circ t_1^{\mathbb{B}}$$

1.2 Robustness of Binarised neural network

There are two types of robustness problems. The qualitative robustness is a problem to determine whether the neural network gives the same (true) output for all inputs in some input region. The quantitative robustness on the other hand determines the part of this input region, which has the same output as some chosen input of this region.

Further I provide formal definition of robustness on functions in general. While I provide multiple types of robustness, in the implementation I will be using only the quantitative robustness for discrete input region (Definition 1.2.1) with constant weight $w(i) = 1$ and strict evaluation function $\bar{h}_p$ (Definition 1.2.3) due to limitations of Clingo framework.

1.2.1 Definition of robustness

Definition 1.2.1 (Quantitative robustness of function). Let F be a function, $F : P \rightarrow Q$. Let I be an *input region* of F , that is $I \subseteq P$. Let w be a function, $w : P \rightarrow \mathbb{R}$, $\forall x \in I : w(x) > 0$, called *weight function*. Let h_p be a function for some *base input* $p \in P$, $h_p : Q \rightarrow \mathbb{R}$, called *evaluation function*, that satisfies

$$\begin{aligned} h_p(F(p)) &= 0 \\ \forall x \in Q. 0 &\leq h_p(x) \leq 1 \end{aligned}$$

Let $Q_{d,h_p}(I)$ be equal to

$$Q_{w,h_p}(I) = \frac{\int_I w(i) \cdot h_p(F(i)) di}{\int_I w(i) di} \quad \text{or} \quad Q_{w,h_p}(I) = \frac{\sum_{i \in I} w(i) \cdot h_p(F(i))}{\sum_{i \in I} w(i)}$$

if the input region is non-discrete or discrete respectively. Then $Q_{w,h_p}(I)$ is called the *quantitative robustness* of function F with a weight function w and evaluation function h_p on input region I .

For empty input region $I = \emptyset$ or non-discrete input regions I such that $\int_I w(i) di = 0$, quantitative robustness is defined to be 0.

The quantitative robustness is an average of values acquired by applying the evaluation function h_p on the input region weighted by the weight function w . *Function highly robust on some input region will have low value of quantitative robustness for this region. The quantitative*

robustness corresponds to the portion of inputs being adversarial on this region.

The weight function can be used to make part of the inputs more prominent. For instance, by the use of weight function such as $w(x) = \frac{1}{1+||p-x||}$, inputs closer to the base input will have higher weight.

The simplest example of the weight function is a constant function $w_1(x) = 1$. With constant function, every input is assigned the same weight, which leads to the quantitative robustness being an average of evaluation function h_p applied over all inputs from the input region.

More complex weight functions can be used to give some areas of the input region higher priority. Such weight function may reflect requirement for the robustness closer to the base input p .

The evaluation function can be used to encode dissimilarity of an input from the base input p . That could prove useful in case of external metric. For instance, if evaluating function F on real numbers against some other function G , following metric might be used:

$$h(x) = \begin{cases} 0 & |F(x) - G(x)| < 1 \\ \log(1/|F(x) - G(x)|) & \text{else} \end{cases}$$

To further familiarize with the notation of quantitative robustness, let me here show you a few properties of this notation.

Lemma 1.2.1. *For input region consisting only of the base input $I = \{p\}$, the quantitative robustness is equal to 0*

Proof.

$$Q_{w,h_p}(\{p\}) = \frac{w(p) \cdot h_p(p)}{w(p)} = \frac{w(p)}{w(p)} \cdot 0 = 0$$

□

Lemma 1.2.2. *Quantitative robustness is between 0 and 1 for every possible combination of functions and input regions.*

Proof. Let me first show that the quantitative robustness is nonnegative. Both w and h_p are nonnegative on P and Q respectively, thus $w(x) \cdot h_p(y)$ is nonnegative for all $(x, y) \in P \times Q$. For that also values of integrals and summations over both $w(x)$ and $w(x) \cdot h_p(y)$ are nonnegative. The fraction of two nonnegative values is nonnegative and the quantitative robustness is nonnegative.

By definition holds $0 \leq h_p(y) \leq 1$, thus also $0 \leq h_p(F(x)) \leq 1$. As $w(x) \geq 0$,

$$0 = w(x) \cdot 0 \leq w(x) \cdot h_p(F(x)) \leq w(x) \cdot 1$$

Again, integration or summation over any subset $I \subseteq P$ can be applied onto the latter two expressions.

$$\begin{aligned} 0 &\leq \int_I w(i) \cdot h_p(F(i)) \, di \leq \int_I w(i) \, di \\ 0 &\leq \sum_{i \in I} w(i) \cdot h_p(F(i)) \leq \sum_{i \in I} w(i) \end{aligned}$$

If the right-hand side term is equal to 0, the statement holds by definition of qualitative robustness. Else by division by the right-hand side term (nonnegative), following holds

$$\begin{aligned} 0 &\leq \frac{\int_I w(i) \cdot h_p(F(i)) \, di}{\int_I w(i) \, di} \leq 1 \\ 0 &\leq \frac{\sum_{i \in I} w(i) \cdot h_p(F(i))}{\sum_{i \in I} w(i)} \leq 1 \end{aligned}$$

□

While the quantitative robustness does tell us something about how much do inputs of the input region differ, the qualitative robustness does only say if there is any wrong output. This may seem less useful, however it can lead to lower computational expenses.

Definition 1.2.2 (Qualitative robustness of function F on input region I). Function $F : P \rightarrow Q$ is (qualitatively) robust on input region $I \subseteq P$ with respect to evaluation function h_p if and only if $Q_{w,h_p}(I) = 0$.

Lemma 1.2.3. *The property of qualitative robustness is independent of the function w .*

Proof. For empty input region, the lemma holds trivially by definition.

Otherwise as weight function w is by definition positive, all statements of the following chain are equivalent.

$$Q_{w,h_p}(I) = \frac{\sum_{i \in I} w(i) \cdot h_p(F(i))}{\sum_{i \in I} w(i)} = 0$$

Since $\sum_{i \in I} w(i) > 0$:

$$\sum_{i \in I} w(i) \cdot h_p(F(i)) = 0$$

As both w and h_p are non-negative

$$\forall i \in I. w(i) \cdot h_p(F(i)) = 0$$

$$\forall i \in I. w(i) = 0 \vee h_p(F(i)) = 0$$

From the definition $\forall x \in P. w(x) > 0$

$$\forall i \in I. h_p(F(i)) = 0$$

This statement is independent of the function w , thus the lemma holds for the discrete variation.

The non-discrete variant can be proven similarly. $\square$

The robustness according to the introduced definition can be contraintuitive when applied to functions that are not continuous on the input region. An extreme example of such function is the Dirichlet function $D : \mathbb{R} \rightarrow \mathbb{R}$.

$$D(x) = \begin{cases} 1 & \text{if } x \in \mathbb{Q} \\ 0 & \text{if } x \in \mathbb{R} \setminus \mathbb{Q} \end{cases}$$

Lemma 1.2.4. *The Dirichlet function D is robust on any interval $\langle k, l \rangle$ where k, l are rational numbers, $k \neq l$.*

Proof of this lemma is attached as Lemma A.1.1.

Since this thesis focuses on robustness of discrete input regions, I will further assume only discrete version of the robustness problem.

Definition 1.2.3 (Strict qualitative robustness of function F on input region I). Function $F : P \rightarrow Q$ is strictly (qualitatively) robust on input region $I \subseteq P$ if and only if for some $\bar{p} \in P$ it is robust on this region with respect to evaluation function $\bar{h}_p$ defined as follows:

$$\bar{h}_p(q) = \begin{cases} 0 & F(\bar{p}) = q \\ 1 & F(\bar{p}) \neq q \end{cases}$$

Lemma 1.2.5. *Function F is strictly robust on input region I if and only if for each two inputs $i, j \in I$, the function F assigns the same value to them i.e. $F(i) = F(j)$.*

Proof. Proof of equivalence in this lemma is done by proving corresponding implications.

For empty input region $I = \emptyset$, the statement holds trivially. Further in the proof I will always assume nonempty input region.

First the left-to-right implication. Let function F be strictly robust on input region I , $\bar{h}_p$ being the evaluation function. As shown in the proof of Lemma 1.2.3, for all instances from input region $i \in I$ holds

$$\bar{h}_p(F(i)) = 0$$

By definition of evaluation function $\bar{h}_p$ it also holds

$$F(p) = F(i)$$

As this holds for every input $i \in I$, for every two inputs $i, j \in I$:

$$F(i) = F(p) = F(j)$$

Now for the opposite implication: Let for every $i, j \in I$, $F(i) = F(j)$. Let $p \in I$. As $\bar{h}_p(F(p)) = 0$ and for every input $i \in I$ it holds that $F(i) = F(p)$, $\bar{h}_p(F(i)) = 0$ for every input from the input region I . The F is thus strictly robust on input region I . $\square$

The strict robustness is equivalent to the robustness as defined in [9]. The t -target robustness from this article is equivalent to my definition of robustness with respect to evaluation function $h_t : Q \rightarrow \mathbb{R}$

$$h_t(q) = \begin{cases} 0 & \text{if } q \neq t \\ 1 & \text{if } q = t \end{cases}$$

Finally the term $Pr(R(u, \tau))$ is equal to quantitative robustness with respect to weight function w_1 and evaluation function h_u on input region $R(u, \tau)$.

1.2.2 Definition of input regions

Both the qualitative and the quantitative robustness rely on subsets of feasible inputs. To be useful when working with binarised neural networks, they such input regions should operate on vectors.

I provide definition of two classes of input regions, input regions based on the Hamming distance and input regions with fixed indices. The input region based on the Hamming distance $R(\vec{u}, r)$ contains all input vectors that difer on at most r positions. The input region with fixed indices (bits) $R(\vec{u}, I)$ specifies set of indices I on which the input vector may not differ form I . Definition are taken from [1].

Definition 1.2.4 (Input region based on the Hamming distance). For an input $\vec{u} \in \mathbb{B}_{\pm 1}^{n_1}$ and an integer $r \geq 0$, let $R(\vec{u}, r) := \{\vec{x} \in \mathbb{B}_{\pm 1}^{n_1} \mid HD(\vec{x}, \vec{u}) \leq r\}$, where $HD(\vec{x}, \vec{u})$ denotes the Hamming distance between $\vec{x}$ and $\vec{u}$.

Intuitively, $R(\vec{u}, r)$ includes input vectors that differ from $\vec{u}$ on at most r positions. Examples of such input regions are:

$$R((1, 1, 1, 1), 1) = \{(1, 1, 1, 1), \\ (-1, 1, 1, 1), (1, -1, 1, 1), (1, 1, -1, 1), (1, 1, 1, -1)\}$$

$$R((-1, 1, -1), 2) = \{(-1, 1, -1), \\ (1, 1, -1), (-1, -1, -1), (-1, 1, 1), \\ (-1, -1, 1), (1, 1, 1), (1, -1, -1)\}$$

As stated at the beginning of this section, in the implementation part, I will only use constant weight function $w_1(i) = 1$. For this specific weight function and the expression in the divisor of quantitative robustness holds that:

$$\sum_{i \in I} w_1(i) = \sum_{i \in I} 1 = ||I||$$

To quickly compute this sum, an explicit formula will be needed.

Lemma 1.2.6. Let $\vec{u}$ be a vector from $\mathbb{B}_{\pm 1}^{n_1}$. Then $||R(\vec{u}, r)|| = \sum_{i=0}^{\min(r, n_1)} \binom{n_1}{i}$

Proof. $R(\vec{u}, r)$ is union of sets

$$R(\vec{u}, r) = \bigcup_{i=0}^r \{\vec{x} \mid HD(\vec{x}, \vec{u}) = i\}$$

These sets are disjoint as elements of each have different number of positions changed. The size of each of these sets is equal to

$$||\{\vec{x} \mid HD(\vec{x}, \vec{u}) = i\}|| = \binom{n_1}{i}$$

because they consist of n_1 positions, out of which i are chosen to be altered. Finally, as for i larger than n_1 , $\binom{n_1}{i} = 0$, the statement holds. $\square$

Definition 1.2.5 (Input region based on fixed bits). For an input $\vec{u} \in \mathbb{B}_{\pm 1}^{n_1}$ and set of indices $I \subseteq \{1, \dots, n_1\}$, let $R(\vec{u}, I) := \{\vec{x} \in \mathbb{B}_{\pm 1}^{n_1} \mid \forall i \in I. x_i = u_i\}$.

The input region based on fixed bits $R(\vec{u}, I)$ does specify the positions which are fixed to the values of the base vector $\vec{u}$. Examples of such input regions are:

$$R((1, 1, -1, 1), \{1, 3, 4\}) = \{(1, 1, -1, 1), (1, -1, -1, 1)\}$$

$$R((1, 1, 1, 1), \{3\}) = \{(1, 1, 1, 1), (-1, 1, 1, 1), (1, -1, 1, 1), (-1, -1, 1, 1), (1, 1, 1, -1), (-1, 1, 1, -1), (1, -1, 1, -1), (-1, -1, 1, -1)\}$$

Lemma 1.2.7. Let $\vec{u}$ be a vector from $\mathbb{B}_{\pm 1}^{n_1}$, $I \subseteq \{1, \dots, n_1\}$. Then $||R(\vec{u}, I)|| = 2^{d-||I||}$

Proof. Each element of I does fix a single position in the input vector. The number of variable positions is $d - ||I||$, each being instance of $+1$ or -1 . This makes the number of variants $2^{d-||I||}$. $\square$

Remark. Lemmas 1.2.6 and 1.2.7 allow for fast computation of the size of the input region.

2 Answer set programming

Answer set programming (ASP) is a form of declarative programming oriented towards difficult, primarily NP-hard, search problems [10]. ASP is particularly suited for solving difficult combinatorial search problems [11]. ASP is somewhat closely related to propositional satisfiability checking (SAT) in sense that the problem is represented as logic program. Difference is in computational mechanism of finding solution.

I will be using framework Clingo [13], an integrated framework for solving ASP programs, for the implementation part of this thesis. As ASP is significantly different from commonly used programming languages, this section should introduce the readers to this paradigm and familiarize them with this different representation of knowledge. Each concept is first defined and then shown on examples.

Should there be ambiguity, please refer to [19] for more examples.

2.1 Extended logic program

Extended logic program is a finite set of facts, rules and constraints each consisting literals. A literal is an atom or its negation. An atom is the elementary construct for representing knowledge [20]. Each atom constitutes a single variable, it can be seen as a possible feature of solution.

Answer set programming differentiates between a query which does not succeed (an atom that is searched for is not derived) and a query which directly fails (the direct negation of the atom is derived). In order to distinguish between these queries, extended logic programs allow for the *classical negation* $\neg$ (direct negation of the atom) in addition to the *negation-as-failure not* (the atom not residing in the answer set) [12].

Further I provide formal definition of extended logic programs according to [12, 19]. For the definition of logic programs I use the notation of a reduct of program as it seems the simplest and is being used for the implementation of Clingo framework. There are however many other equivalent definitions [21].

2.1.1 Syntax of Extended logic program

Definition 2.1.1 (Signature [22]). A (many-sorted) signature is a tuple $\Sigma = (S, C, F, P)$, where:

- S is a nonempty set of sorts, each sort $\sigma \in S$ is associated with a set of variables $\mathbf{Var}_\sigma$ and set of constants $\mathbf{Con}_\sigma$,
- C is a countable set of constant symbols whose sorts belong to S ,
- F is a countable set of function symbols whose arities are constructed using sorts that belong to S ,
- P is a countable set of predicate symbols whose arities are constructed using sorts that belong to S .

Given a signature $\Sigma = (S, C, F, P)$, we write Σ^S for S , Σ^C for C , Σ^F for F and Σ^P for P .

Definition 2.1.2 (Terms [22]). Let Σ be a signature. The set of Σ -terms of sort σ is the smallest set of expressions satisfying the following properties:

- Each variable x of sort σ is a term of sort σ , provided that $\sigma \in \Sigma^S$.
- Each constant symbol $c \in \Sigma^C$ of sort σ is a Σ -term of sort σ .
- If $f \in \Sigma^F$ is a function symbol of arity $\sigma_1 \times \cdots \times \sigma_n \rightarrow \sigma$ and t_i is a Σ term of sort σ_i , for $i = 1, \dots, n$, then $f(t_1, \dots, t_n)$ is a term of sort σ .

Definition 2.1.3 (Atoms, literals [22]). Let Σ be a signature. A Σ -atom is an expression of the form

$$p(t_1, \dots, t_n)$$

where $p \in \Sigma^P$ is a predicate symbol of arity $\sigma_1 \times \cdots \times \sigma_n$ and for $i = 1, \dots, n$, t_i is a Σ -term of sort σ_i .

A Σ -literal is a formula of the form

$$\varphi \text{ or } \neg\varphi$$

where φ is a Σ -atom.

For both function and predicate symbols, infix notation can be used if it was established (e.g. $+$, $-$, $=$, $\leq$, $\dots$). If such notation would result in ambiguity, brackets should be used.

Example 2.1.1. Let Σ_1 be a signature with sorts

$$\sigma_{\mathbb{B}} = \{True, False\}, \sigma_{\mathbb{N}} = \mathbb{N}_0$$

and with function symbols $\vee, \wedge$ of arity $\sigma_{\mathbb{B}} \times \sigma_{\mathbb{B}} \rightarrow \sigma_{\mathbb{B}}$, and function symbols $=, <$ of arity $\sigma_{\mathbb{N}} \times \sigma_{\mathbb{N}} \rightarrow \sigma_{\mathbb{B}}$. Let p be a predicate of arity $\sigma_{\mathbb{B}}$, and q be a predicate of arity $\sigma_{\mathbb{N}} \times \sigma_{\mathbb{B}}$. Also let variables X, Y, Z be of sort $\sigma_{\mathbb{N}}$ and let there be no variables of sort $\sigma_{\mathbb{B}}$.

Then expressions

$$\begin{aligned} &True, False, \\ &True \vee False, False \wedge True, \\ &(42 = 42), (X = 17), (3 < Y) \vee (6 = 6), \\ &[(5 = 1) \wedge True] \vee False \end{aligned}$$

are some of the Σ -terms of sort $\sigma_{\mathbb{B}}$.

Expressions

$$\begin{aligned} &p(True), p(False), \\ &p([(5 = 1) \wedge True] \vee False), \\ &q(0, True), q(42, False), \\ &q(123, (42 = 42)), q(93, True \vee (Z < 17)) \end{aligned}$$

are some of the Σ -atoms.

Definition 2.1.4 (Rule [19]). Let Σ be a signature. A Σ -rule (Σ -formula) r is an expression of the form

$$l_0 \text{ or } \dots \text{ or } l_k \leftarrow l_{k+1}, \dots, l_m, \text{not } l_{m+1}, \dots, \text{not } l_n.$$

where each l_i is a Σ -literal. The following notation is used:

$$\begin{aligned} \text{head}(r) &= \{l_0, \dots, l_k\} \\ \text{body}(r) &= \{l_{k+1}, \dots, l_m, \text{not } l_{m+1}, \dots, \text{not } l_n\} \end{aligned}$$

$$\text{body}^+(r) = \{l_{k+1}, \dots, l_m\}$$

$$\text{body}^-(r) = \{\text{not } l_{m+1}, \dots, \text{not } l_n\}$$

Further, if $\text{head}(r) = \emptyset$, the rule is called a constraint and is written as

$$\leftarrow l_{k+1}, \dots, l_m, \text{not } l_{m+1}, \dots, \text{not } l_n.$$

If $\text{body}(r) = \emptyset$, the rule is called a fact and is written as

$$l_0 \text{ or } \dots \text{ or } l_k.$$

The term “not ” is called default negation.

Definition 2.1.5 (Logic program [19]). Logic program is a pair (Σ, Π) where Σ is a signature and Π is a collection (set) of Σ -rules.

Example 2.1.2. An example of a logic program with signature Σ_1 from Example 2.1.1 is a pair (Σ_1, Π_1) where Π_1 is a set

$$\Pi_1 = \begin{cases} p(\text{True}). \\ q(5, \text{True}). \\ p(\text{False}) \leftarrow \text{not } p(\text{True}). \\ q(Y, \text{True}) \leftarrow q(X, (Y < X)). \end{cases}$$

Logic program is often denoted only by its second element Π . In that case its signature consists of symbols occurring in the program.

In this thesis, I will be mostly working with logic programs that have zero or one literal in the head of each of their rules. While allowing for multiple literals in the head of a rule may allow for otherwise impossible to achieve expressions, they are less readable and have a higher computational cost. General logic programs are in the complexity class Σ_2^P [23], logic programs without disjunction in head are *NP*-complete [24] and logic programs with neither disjunction nor default negation belong to *P* [19].

2.1.2 Semantics of extended logic program

For the definition of the extended logic programs to be usefull, definition of stable models is needed. Many definitions have been created [21]. Here I will show the definition using the reduct of a logic

program [19], which was used for the ASP solver I am using for the implementation of verification, Clingo [25].

First of all, the logic program needs to be grounded. Grounding is a process in which we substitute each rule in the program for its equivalent rules with constant symbols only.

The semantics of function symbols is usually defined in the grounder (the program that does the grounding) thus the programs can be grounded before the start of solving of the logic program.

Definition 2.1.6 (Ground program [19]). Terms, literals, and rules of program Π with signature Σ are called ground if they contain no variables and no function symbols. A program is called ground if all its rules are ground. A rule r' is called a ground instance of a rule r of Π if it is obtained from r by:

- replacing r 's variables by properly typed ground terms of Σ ;
- replacing r 's function terms by their values.

A program $gr(\Pi)$ consisting of all ground instances of all rules of Π is called the ground instantiation of Π .

For an example of grounding of logic program (Σ_1, Π_1) , see Example A.2.1.

Grounders differ in the exact algorithm of the grounding as well as in the allowed signature of the logic program. The exact full syntax and semantics of the ASP grounder Gringo can be found in [26].

Definition 2.1.7 (Partial interpretation of a signature). A partial interpretation S of a signature Σ is a set of Σ -literals in which for each Σ -atom φ there is either φ , $\neg\varphi$ or neither of them.

A partial interpretation can be used to denote the solution of a logic program. It allows for 3-valued logic — an atom either is true, is false or is unknown. The atom being unknown can specify an atom, that was not derived, while the atom being false strictly says it cannot be true (its negation has been derived).

For a partial interpretation to be a solution (model, answer set) of the program, it further needs to be consistent with the logic program and all its literals have to be founded in the program (the set has to be minimal).

Each rule r is seen as a statement “head(r) holds if all the literals from the body⁺(r) hold and at the same time no literal from the body⁻(r) hold”. The facts have empty body, thus their head has to hold in every solution. On the other hand, the body of constraint can never be consistent with the solution as its head is empty thus can not be consistent with solution. Further follows a more formal definition.

Definition 2.1.8 (Consistent partial interpretation).

- Partial interpretation S is said to be consistent with the body of rule r if and only if $\text{body}^+(r) \subseteq S$ and $\text{body}^-(r) \cap S = \emptyset$.
- Partial interpretation S is said to be consistent with the head of rule r if and only if $\text{head}(r) \cap S \neq \emptyset$
- Partial interpretation S is said to be consistent with a rule r if and only if S is not consistent with body(r) or S is consistent with head(r).
- Partial interpretation S is said to be consistent with a logic program (Σ, Π) if and only if it is consistent with all of its rules.

Definition 2.1.9 (Basic logic program). A logic program (Σ, Π) is called basic if there is no negative atom in body of any of its rules nor constraints, that is

$$\forall r \in \Pi. \text{body}^-(r) = \emptyset$$

Now for the answer sets (partial interpretations that are solutions) of a logic program a reasonable constraints on them are to satisfy (be consistent with) the logic program and to only contain information that needs to be true (each answer set has to be minimal).

Definition 2.1.10 (Answer set of a basic logic program [19]). Let (Σ, Π) be a basic logic program and S be a partial interpretation of Σ . S is an answer set (solution) for Π if S is minimal (in the sense of set-theoretic inclusion) among the partial interpretations consistent with Π .

For a grounded basic logic program, finding its answer set (solution) is easy. It can be built incrementally. Heads of facts have to be included in the partial interpretation for it to be an answer set. Then in each step all heads of rules, that have its bodies consistent with the

previous set, are added into the set. This is done as long as any literal is added into the partial interpretation.

Lemma 2.1.1. *Incremental building of the answer set of the basic logic program. Let (Σ, Π) be a basic logic program with at most one literal in the head of every rule. Let*

$$S_0 = \emptyset,$$

$$S_{i+1} = S_i \cup \{\varphi \mid \{\varphi\} = \text{head}(r), \text{body}^+(r) \subseteq S_i\} \text{ for each } i \geq 0.$$

Denote by S_∞ the union of all S_i for $i \geq 0$. If it holds that:

- for every Σ -atom φ , S_∞ contains at most one of literals φ , $\neg\varphi$, and
- S_∞ is consistent with every constraint of Π ,

then S_∞ is the only answer set of program (Σ, Π) . Else (Σ, Π) has no answer set.

For a proof of this lemma, see Lemma A.1.2.

Example 2.1.3. Let (Σ, Π) be a logic program,

$$\Pi = \begin{cases} p(a). \\ p(b) \leftarrow p(a). \\ p(c) \leftarrow p(b). \\ p(x). \\ p(y) \leftarrow p(x), p(a). \\ p(z) \leftarrow p(x), p(c). \\ p(n) \leftarrow p(a), p(k). \end{cases}$$

When building the solution, we first start with an empty set.

$$S_0 = \{\}$$

In the first step only the facts have bodies consistent with the partial solution S_0 . We can add literals in their heads into the partial solution.

$$S_1 = \{p(a), p(x)\}$$

With these two literals in the partial solution, rules $p(b) \leftarrow p(a).$ and $p(y) \leftarrow p(x), p(a).$ have their bodies consistent with the partial

solution S_1 . For this we add literals in heads of these rules into the partial solution.

$$S_2 = \{p(a), p(x), p(b), p(y)\}$$

In the next step, another rule $p(c) \leftarrow p(b)$. has its body consistent with the partial solution S_2 . We add the literal from its head into the partial solution.

$$S_3 = \{p(a), p(x), p(b), p(y), p(c)\}$$

This addition into the partial solution makes the body of yet another rule $p(z) \leftarrow p(x), p(c)$. to be consistent with the partial solution.

$$S_4 = \{p(a), p(x), p(b), p(y), p(c), p(z)\}$$

No other literals have to be added into the partial solution. S_4 is the only solution of logic program (Σ, Π) .

The only literal from the heads of rules that have not been included into the partial solution is $p(n)$. This literal relies on some unknown literal $p(k)$ that is not in any head of rules of Π . Another case of literal that is not in a solution of logic program is shown in the following example.

Example 2.1.4. Let (Σ, Π) be a logic program,

$$\Pi = \begin{cases} p(a) \leftarrow p(b). \\ p(b) \leftarrow p(a). \end{cases}$$

This logic program a single solution, that is an empty set. While the partial interpretation $\{p(a), p(b)\}$ is consistent with the logic program, it is not the minimal set.

The simple way of building answer sets of basic programs, can be extended to all logic programs. In basic programs we have assumed that no rule with default negation exists in the logic program. To fight the default negation in a general extended logic program, we can first define a partial interpretation of Σ that constitutes the possible answer set, then take all rules $r \in \Pi$ that have no intersection of their body⁻(r)

with our partial interpretation and into the reduced logic program include only the nonnegative body⁺(r) from them. If this reduced program yields the same answer set as the partial interpretation we have defined, it is an answer set.

Definition 2.1.11 (Reduct of a logic program [19]). Let (Σ, Π) be a logic program, S be a partial interpretation of Σ . Let Π^S be a set of rules such that

$$\Pi^S = \{\text{head}(r_i) \leftarrow \text{body}^+(r_i). \mid r_i \in \Pi, \text{body}^-(r_i) \cap S = \emptyset\}$$

Then Π^S is called a reduct of Π relative to the partial interpretation S .

In the reduct can be encountered a new type of rule that was not yet discussed here.

Example 2.1.5. Let $\Pi = \{\leftarrow \text{not } \varphi.\}$, $S = \{\}$ for some atom φ .

The reduct Π^S contains a single rule with both head and body empty. Based on the Definition 2.1.8, no partial interpretation can be consistent with such rule as no partial interpretation can be consistent with an empty head and every partial interpretation is consistent with an empty body. This is however a desired behaviour, the interpretation S was defined in a way it was not consistent with the rule so it is only right the reduct can not have any answer set.

Definition 2.1.12 (Answer set of general grounded logic program [19]). A partial interpretation S of Σ is an answer set for (Σ, Π) if S is an answer set for Π^S .

Definition 2.1.12 gives a direct way of computing answer sets of any logic programs. For every possible subset S of grounded atoms in program Π , the reduct Π^S can be constructed and evaluated for its answer set. If the found answer set is equal to the subset of grounded atoms S , it is said to be also an answer set of the program Π .

Let's illustrate the computation of an answer set of general logic program on two examples.

Example 2.1.6. Let (Σ, Π) , where

$$\Pi = \{\varphi \leftarrow \varphi., \psi \leftarrow \text{not } \varphi.\},$$

be a logic program with two atoms φ, ψ . There are 4 subsets of set of all atoms. First the reduct relative to the subset is made, on it the calculation of its answer set (AS) is made. If the answer set of reduct is equal to the subset, it is an answer set of the whole logic program.

S	Π^S	AS of Π^S
$\{\}$	$\varphi \leftarrow \varphi.$ $\psi \leftarrow .$	$\{\psi\}$
$\{\varphi\}$	$\varphi \leftarrow \varphi.$	$\{\}$
$\{\psi\}$	$\varphi \leftarrow \varphi.$ $\psi \leftarrow .$	$\{\psi\}$
$\{\varphi, \psi\}$	$\varphi \leftarrow \varphi.$	$\{\}$

There is only a single answer set of (Σ, Π) , that is $\{\psi\}$. Similiar to Example 2.1.4, in the reduct $\Pi^{\{\varphi\}}$ partial interpretation $\{\varphi\}$ is not an answer set as it is not minimal.

Example 2.1.7. Let (Σ, Π) , where

$$\Pi = \{\varphi \leftarrow \text{not } \psi., \psi \leftarrow \text{not } \varphi.\},$$

be a logic program with two atoms φ, ψ . Again, there are 4 subsets of set of all atoms.

S	Π^S	AS of Π^S
$\{\}$	$\varphi \leftarrow .$ $\psi \leftarrow .$	$\{\varphi, \psi\}$
$\{\varphi\}$	$\varphi \leftarrow .$	$\{\varphi\}$
$\{\psi\}$	$\psi \leftarrow .$	$\{\psi\}$
$\{\varphi, \psi\}$		$\{\}$

This time there are two answer sets of (Σ, Π) , $\{\varphi\}$ and $\{\psi\}$.

2.2 Clingo

Clingo is an integrated ASP system, consisting of a grounder Gringo and solver Clasp [27].

If you are not familiar with the Clingo framework, see Section B.1, where I show basics of the Clingo language, Gringo as translation from Clingo to Aspif language and solving such program with Clasp.

2.2.1 Clingo language

Clingo language [13] is used for transcribing ASP programs in the Clingo system. Clingo language does allow for facts, rules and constraints just like the logic programs defined in this thesis. Clingo language is in fact almost identical to the already provided notation.

2.2.2 Gringo

Gringo [25] is software that grounds logic program in Clingo language and translates it into the Aspif format that is readable by logic program solver Clasp. Gringo first resolves every rule with variables into (possibly multiple) variable-free rules and then changes format of the program into Clasp-readable Aspif. Gringo thus can introduce new atoms that were not obvious from the Clingo program.

2.2.3 Clasp

Clasp is the solver of the Clingo framework. It takes logic program in the Aspif format to search for answer sets of a grounded logic program. The solver approaches the inference using the unit propagation of nogoods [28].

For the inference of answer sets of some logic program, Clasp uses backpropagation over a tree of partial solutions. Each time it sees a partial solution that is not an answer set of the program, it derives new constraints on the solution which it propagates up the tree of partial solutions. This allows Clasp to efficiently prune large branches of partial solutions.

For model (answer sets) counting, Clasp uses a method called model enumeration. To determine that a set is an answer set, it needs

to evaluate the exact set. This means, that the time complexity of finding all answer sets is always at least linear with respect to the number of answer sets.

3 ASP encoding of BNN robustness

In this section I show possible encodings of binarised neural networks and of the robustness problem. Specifically, I create an encoding of a binarised neural network using negation as failure, input regions and adversarial inputs. Finally, I create a Python program able to encode weights and biases of a binarised neural network into a part of Clingo program.

3.1 Analysis of BNN

In the Definition 1.1.4 I have introduced a binarised perceptron $p^{\mathbb{B}}$ as a linear model composed from an inner potential ξ , parametrised by real bias b and vector of binarised values $\vec{w}$, and a heavyside step function H . Further I have shown that this model has the exact same expressive power as the binarised perceptron with batch normalization.

Then in the Appendix B I have shown the Clingo language. This language allows for computations over the set of whole numbers. It also allows for the use of aggregate expressions, which can be leveraged for summations.

The last large roadblock in the encoding of binarised perceptron into Clingo language is the real bias, as Clingo does only allow for the use of whole numbers.

Lemma 3.1.1. *For every binarised perceptron with bias from real numbers there is an equivalent binarised perceptron with bias from whole numbers and vice versa.*

Proof. Idea behind this construction comes from [1].

Let $p^{\mathbb{B}}$ be a binarised perceptron without batch normalisation.

$$p^{\mathbb{B}}(\vec{x}) = H \circ \xi(\vec{x}) = \begin{cases} 1 & b + \sum_{i=1}^k w_i \cdot x_i \geq 0 \\ -1 & b + \sum_{i=1}^k w_i \cdot x_i < 0 \end{cases}$$

Both each w_i and x_i are whole numbers (± 1 -binarised values), thus also the sum $\sum_{i=1}^k w_i \cdot x_i$ is a whole number. As the formula for binarised perceptron only compares a sum of a real number and a whole

number to a whole number, in both cases integer part of the bias $\lfloor b \rfloor$ can be used in place of the bias.

$$p^{\mathbb{B}}(\vec{x}) = H \circ \zeta(\vec{x}) = \begin{cases} 1 & \lfloor b \rfloor + \sum_{i=1}^k w_i \cdot x_i \geq 0 \\ -1 & \lfloor b \rfloor + \sum_{i=1}^k w_i \cdot x_i < 0 \end{cases}$$

Such perceptron uses only whole numbers. Whole numbers are subset of real numbers, thus also the implication in the other direction is true. $\square$

Remark. The proof of Lemma 3.1.1 gives a direct way to encode a binarised perceptron without batch normalisation into a binarised perceptron with bias from whole numbers. Using also the Lemma 1.1.2, a binarised perceptron with bias from whole numbers can be easily constructed even from binarised perceptron with batch normalisation.

Real bias is however included not only in perceptrons of inner layers, but also in the last Argmax layer. A transcription of this type of layer into whole numbers is needed.

Lemma 3.1.2. *Every Argmax layer with real-valued bias parameters can be transcribed using multiple whole numbers in place of bias parameters.*

Proof. The Argmax layer $t^{am} : \mathbb{B}^m \rightarrow \mathbb{B}^n$ consists of inner potential ζ and argmax encoding using the one-hot vector.

$$\begin{aligned} t^{am}(\vec{x}) &= (y_1, \dots, y_n) \\ y_k &= \begin{cases} 1 & k = \arg \max_{i=1}^n (\zeta_i(\vec{x})) \\ 0 & \text{otherwise} \end{cases} \\ \zeta_i(\vec{x}) &= b_i + \sum_{j=1}^m w_{i,j} \cdot x_j \end{aligned}$$

The condition of an output position y_k having value 1 can be rewritten into another condition using the maximality of this position.

Position y_k on the output has value of 1 if and only if for every other output position y_l one of following holds:

- $\zeta_k(\vec{x}) > \zeta_l(\vec{x})$

- $\xi_k(\vec{x}) = \xi_l(\vec{x}) \wedge k < l$

By substitution for inner potential and splitting bias b into its integer part $\lfloor b \rfloor$ and fractional part $\{b\}$, the first condition splits. The condition can be true either if the integer part $\lfloor \xi_k(\vec{x}) \rfloor$ is bigger than the integer part $\lfloor \xi_l(\vec{x}) \rfloor$ or if they are equal and the fractional part $\{\xi_k(\vec{x})\} = \{b_k\}$ is bigger than $\{\xi_l(\vec{x})\} = \{b_l\}$:

- $\lfloor b_k \rfloor + \sum_{j=1}^m w_{k,j} \cdot x_j > \lfloor b_l \rfloor + \sum_{j=1}^m w_{l,j} \cdot x_j$
- $\lfloor b_k \rfloor + \sum_{j=1}^m w_{k,j} \cdot x_j = \lfloor b_l \rfloor + \sum_{j=1}^m w_{l,j} \cdot x_j \wedge \{b_k\} > \{b_l\}$
- $\lfloor b_k \rfloor + \sum_{j=1}^m w_{k,j} \cdot x_j = \lfloor b_l \rfloor + \sum_{j=1}^m w_{l,j} \cdot x_j \wedge \{b_k\} = \{b_l\} \wedge k < l$

Conditions in the last paragraph do provide for an encoding using whole numbers. Integer parts of biases b_k and b_l are already whole numbers. The output positions are enumerated, thus k and l are already whole numbers. Finally, on fractional parts $\{b_k\}$ and $\{b_l\}$ an ordering can be created. The Argmax layer can thus be transcribed using only whole numbers. $\square$

Remark. Instead of using both the ordering of the fractional part of bias $\{b_k\}$ and the position k , an ordering that prioritizes high fractional part of bias and with lower priority low position k can be used. I will be using this ordering in the encoding of BNN.

3.2 ASP encoding of BNN

In the encoding of binarised neural network in Clingo language I use predicate symbols with meaning described in Table 3.1. This table contains atoms with their meaning categorised into ones that encode the binarised neural network, ones that specify the input region and ones that are computed while grounding. Both atoms that define the binarised neural network and that specify the input region are given to the logic program as facts by the Python encoder.

3. ASP ENCODING OF BNN ROBUSTNESS

Symbol	Semantics	
Encoding of BNN		
$\text{layer}(L, N)$	L	Layer number Input layer has number 0 Output layer has the highest number
	N	Number of perceptrons in layer L
	L	Layer
$\text{weight}(L, M, N, W)$	M	Position of perceptron of layer $L - 1$
	N	Position of perceptron in layer L
	W	Weight of input M to perceptron N in layer L
$\text{bias}(L, N, B)$	L	Layer
	N	Position of perceptron in layer L
	B	Bias of perceptron N in layer L
$\text{outpre}(N, P)$	N	Position of output
	P	N has precedence P lower precedence mean higher priority
Encoding of input space		
$\text{input}(N)$		Specification of the base input
	N	Input on the position N of input layer has value 1
$\text{hammdist}(R)$		The input space is based on the hamming distance
	R	Maximal hamming distance from base input is R
$\text{inpfix}(N)$		The input space is based on fixed bits
	N	Position N of input is fixed to the value of base input
Computation		
$\text{output_layer}(L)$	L	Layer number L is the output layer
$\text{potential}(L, N, P)$	L	Layer number
	N	Position of perceptron in layer L
	P	Perceptron N of layer L has inner potential P
$\text{on}(L, N)$	L	Layer number
	N	Perceptron on position N of layer L outputs 1
$\text{output}(N)$	N	Output on position N of output layer has value 1

Table 3.1: Semantics of encoding BNN into Clingo-readable file

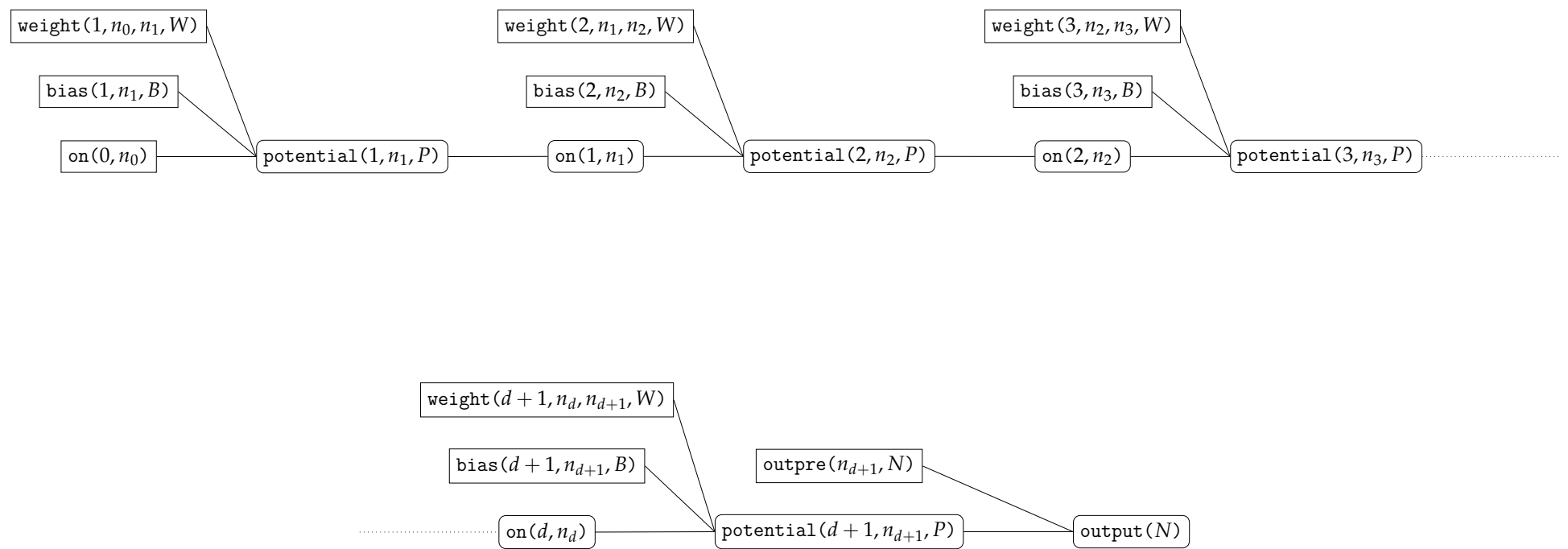


Figure 3.1: Schema of a multi-layer perceptron encoding

The encoding of BNN into Clingo language will be a composition of layers. Output of each layer is dependent only on the output values, weights and biases of the previous layer. Schema of inference for this architecture is shown in Figure 3.1. The schema contains two types of nodes, rectangular nodes do contain data that can be seen as an input of the inference and nodes with rounded corners that are computed in the runtime of the inference. As can be seen from the schema, the inference is linear, meaning the value of each layer is dependent only on the input data or data computed in the previous layer.

3.2.1 Encoding of a perceptron

As shown in Lemma 3.1.1, each binarised perceptron can be encoded using only bias with value from whole numbers. The encoding in Clingo language is straightforward.

```
potential(L, N, S+B) :-
    S = #sum{
        W,M :      on(L-1, M), weight(L, M, N, W);
        -W,M : not on(L-1, M), weight(L, M, N, W) },
    bias(L, N, B).

on(L,N) :- potential(L, N, P), P >= 0.
```

Encoding 1: Encoding of perceptron using potential

This implementation is a direct encoding of a binarised perceptron. It follows the formula of perceptron as described in the Lemma 3.1.1. The implementation is however weak as it relies on the use of intermediate symbol `potential`. While grounding, this would require to build a literal for every possible value of inner potential of each perceptron in the network and consequently in a large ground program as shown in the Example B.2.1. To fight that, literals of symbol `on` may be built directly.

```
on(L, N) :-
    -B <= #sum{
        W,M :      on(L-1, M), weight(L, M, N, W);
        -W,M : not on(L-1, M), weight(L, M, N, W) },
    bias(L, N, B).
```

Encoding 2: Direct encoding of perceptron

The last implementation still has a single flaw. It uses both the positive and negative variant of the literal on $(L-1, M)$ in sense of the default negation. We can however make a transformation of the BNN such that it eliminates use of the negative variant.

Starting with the expression from 3.1.1, the ± 1 -binarised input vector $\vec{x}$ can be substituted by a $\{1, 0\}$ -binarised input vector $\vec{x}_b$:

$$x_{b,i} = \begin{cases} 1 & x_i = 1 \\ 0 & x_i = -1 \end{cases}$$

$$x_i = 2 \cdot x_{b,i} - 1$$

$$\xi(\vec{x}) = b + \sum_{i=1}^k w_i \cdot (2 \cdot x_{b,i} - 1)$$

This expression can be further transformed by splitting the sum and eliminating the multiplication by 2.

$$\xi(\vec{x}) = b + \sum_{i=1}^k w_i \cdot 2 \cdot x_{b,i} - \sum_{i=1}^k w_i \cdot 1$$

$$\xi(\vec{x}) = b - \sum_{i=1}^k w_i + 2 \sum_{i=1}^k w_i \cdot x_{b,i}$$

$$p^B(\vec{x}) = H \circ \xi(\vec{x}) = \begin{cases} 1 & b - \sum_{i=1}^k w_i + 2 \sum_{i=1}^k w_i \cdot x_{b,i} \geq 0 \\ -1 & b - \sum_{i=1}^k w_i + 2 \sum_{i=1}^k w_i \cdot x_{b,i} < 0 \end{cases}$$

Finally, both cases can be divided by 2. The expression $\frac{b - \sum_{i=1}^k w_i}{2}$ in the final form of equality is independent of the input vector. It can be seen as the new bias, thus integer part of this expression can take its place similarly to Lemma 3.1.1.

$$p^B(\vec{x}) = H \circ \xi(\vec{x}) = \begin{cases} 1 & \left\lfloor \frac{b - \sum_{i=1}^k w_i}{2} \right\rfloor + \sum_{i=1}^k w_i \cdot x_{b,i} \geq 0 \\ -1 & \left\lfloor \frac{b - \sum_{i=1}^k w_i}{2} \right\rfloor + \sum_{i=1}^k w_i \cdot x_{b,i} < 0 \end{cases}$$

With weights and biases adjusted in the way described above, the perceptron can be encoded without the use of default negation:

```

on(L, N) :-
    -B <= #sum{ W,M : on(L-1, M), weight(L, M, N, W) },
    bias(L, N, B).

```

Encoding 3: Direct encoding of perceptron using (0,1)-binarised values

3.2.2 Encoding of an Argmax layer

As shown in Lemma 3.1.2 and remark that follows it, Argmax layer can be encoded using the vector of weights, biases and the precedence of individual outputs. First, literals of predicate symbol potential can be constructed similarly to the perceptron, on these max aggregate can be used.

```

potential(D+1, N, S+B) :-
    S = #sum{
        W,M :      on(D, M), weight(D, M, N, W);
        -W,M : not on(D, M), weight(D, M, N, W) },
    bias(D, N, B), output_layer(D+1).

output(N) :-
    (Sum, Precedence, Node) = #max{
        (S, -P, N) : potential(D+1, N, S), outpre(N, P)
    },
    output_layer(D+1).

```

Encoding 4: Encoding of argmax layer using potential and max aggregate

While this may work, in the grounding process it will expand into a large ruleset. The ruleset will contain a rule with potential in head for every possible value of sum of inputs and bias, for each of these multiple rules to implement the max aggregate will be created.

The potential can however not be directly substituted for a comparison in the body of output like in the implementation of perceptron as the value of inner potential is needed for the comparison of different output values.

To overcome this problem, it can be observed from the other side. Instead of asking whether the particular output position has the largest inner potential, all output position with inner potential less than some other can be forbidden from being the final output. An output literal can then be introduced into the solution using a rule with count aggregate in its head.


```

potential(D+1, N, S+B) :-
    S = #sum{
        W,M :      on(D, M), weight(D+1, M, N, W);
        -W,M : not on(D, M), weight(D+1, M, N, W) },
    bias(D+1, N, B), output_layer(D+1).

1 { output(1..N) } 1 :- output_layer(D+1), layer(D+1, N).

:- output(N),
    potential(D+1, N, S), potential(D+1, M, T),
    output_layer(D+1),
    N != M, S < T.
:- output(N),
    potential(D+1, N, S), potential(D+1, M, T),
    outpre(N, P), outpre(M, Q),
    output_layer(D+1),
    N != M, S = T, P > Q.

```

Encoding 5: Encoding of argmax layer using potential and constraints on output

Negated literals with predicate symbol on can be eliminated similarly to the elimination in the encoding of perceptron (see Section 3.2.1). The precedence must be however calculated from the new bias after elimination. (For the implementation see Section 3.3.2.)

```

potential(D+1, N, S+B) :-
    S = #sum{ W,M : on(D, M), weight(D+1, M, N, W) },
    bias(D+1, N, B), output_layer(D+1).

1 { output(1..N) } 1 :- output_layer(D+1), layer(D+1, N).

:- output(N),
    potential(D+1, N, S), potential(D+1, M, T),
    output_layer(D+1),
    N != M, S < T.
:- output(N),
    potential(D+1, N, S), potential(D+1, M, T),
    outpre(N, P), outpre(M, Q),
    output_layer(D+1),
    N != M, S = T, P > Q.

```

Encoding 6: Encoding of argmax layer using potential and constraints on output with (0,1)-binarization

Literals with predicate symbol potential can now be also eliminated by using sum aggregate directly in the comparison of inner potential values.

```

1 { output(1..N) } 1 :- output_layer(D+1), layer(D+1, N).

:- output(N),
   S = #sum{ W,O : on(D, O), weight(D+1, O, N, W) },
   T = #sum{ W,O : on(D, O), weight(D+1, O, M, W) },
   bias(D+1, N, B), bias(D+1, M, C),
   output_layer(D+1),
   N != M, S + B < T + C.

:- output(N),
   S = #sum{ W,O : on(D, O), weight(D+1, O, N, W) },
   T = #sum{ W,O : on(D, O), weight(D+1, O, M, W) },
   bias(D+1, N, B), bias(D+1, M, C),
   outpre(N, P), outpre(M, Q),
   output_layer(D+1),
   N != M, S + B = T + C, P > Q.

```

Encoding 7: Direct encoding of argmax layer using constraints on output and equality

While grounding, the value of an aggregate may be only compared to an integer value. It may not be compared with value of another aggregate directly. This results in a grounding of this logic program being more or less the same as of the last logic program using potential (in fact, the grounded program is even bigger than before). To avert this, both aggregates have to be merged into a single one.

$$S + B < T + C$$

$$S - T < C - B$$

The expression is easily transformed for variables representing values of aggregates to be on one side. As both aggregates are sum aggregates, values of aggregate with variable T may be multiplied by -1. As the computation of aggregate expression uses representation as sets, additional value must be added to tuples for summation to distinguish between values of weights for node N and M. The same transformation may be done also with the second constraint of the logic program.

```

1 { output(1..N) } 1 :- output_layer(D+1), layer(D+1, N).

:- output(N),

```

```

    C - B < #sum{ W,0, 1 : on(D, 0), weight(D+1, 0, N, W
) ;
                                -W,0,-1 : on(D, 0), weight(D+1, 0, M, W
) },
    bias(D+1, N, B), bias(D+1, M, C),
    output_layer(D+1),
    N != M.
:- output(N),
    C - B = #sum{ W,0, 1 : on(D, 0), weight(D+1, 0, N, W
) ;
                                -W,0,-1 : on(D, 0), weight(D+1, 0, M, W
) },
    bias(D+1, N, B), bias(D+1, M, C),
    outpre(N, P), outpre(M, Q),
    output_layer(D+1),
    N != M, P > Q.

```

Encoding 8: Direct encoding of argmax layer using constraints on output and and equality

Finally, as described in Example B.2.1, when an equality symbol is used in an aggregate, while grounding it is transformed into two inequalities. To further simplify (and shorten) the grounded logic program, symbol $\leq$ may be used instead of $=$ in the second constraint. If $C - B < S - T$, then the output is not the right one by the first constraint already, thus it is equivalent to the previous logic program.

```

1 { output(1..N) } 1 :- output_layer(D+1), layer(D+1, N).

:- output(N),
    C - B < #sum{ W,0, 1 : on(D, 0), weight(D+1, 0, N, W
) ;
                                -W,0,-1 : on(D, 0), weight(D+1, 0, M, W
) },
    bias(D+1, N, B), bias(D+1, M, C),
    output_layer(D+1),
    N != M.
:- output(N),
    C - B <= #sum{ W,0, 1 : on(D, 0), weight(D+1, 0, N,
W) ;
                                -W,0,-1 : on(D, 0), weight(D+1, 0, M,
W) },
    bias(D+1, N, B), bias(D+1, M, C),
    outpre(N, P), outpre(M, Q),
    output_layer(D+1),

```

$N \neq M, P > Q.$

Encoding 9: Direct encoding of argmax layer using constraints on output and inequality

3.2.3 Encoding of input regions

Similarly to the final encoding of the argmax layer, I will encode input region by first allowing for any input vector using an head aggregate with unspecified boundaries over literals $\text{on}(0, 1..N)$, on top of this I will then build constraints defined by the desired input region.

Input region based on the Hamming distance

To constrain input such that only inputs with hamming distance at most r are allowed, a simple aggregate rule can be added into the set of rules. The rule denies any partial solution with input that contains more than r positions differing from the base input to be an answer set.

```
{ on(0, 1..N) } :- layer(0, N).
:- #count{ N :      on(0, N), not input(N);
          N : not on(0, N),      input(N) } > R,
   hammdist(R).
```

Encoding 10: Encoding of input region based on Hamming distance

Input region based on the fixed bits

Constraining the input on some fixed bits is trivial. To make such constraint, for each fixed position, one can forbid the base input position having value 1 and real input position not having 1 and vice versa.

```
{ on(0, 1..N) } :- layer(0, N).
:-      on(0, N), not input(N), inpfix(N).
:- not on(0, N),      input(N), inpfix(N).
```

Encoding 11: Encoding of input region based on fixed bits using constraints on input

The second constraint can be substituted for a rule with $\text{on}(0, N)$ in its head and not in its body.

```

{ on(0, 1..N) } :- layer(0, N).
:- on(0, N), not input(N), inpfix(N).
on(0, N) :- input(N), inpfix(N).

```

Encoding 12: Encoding of input region based on fixed bits using rule and constraint

Finally, instead of allowing for any input to have either value in the start, one can encode the possibility for another value only for the input positions that are not fixed. This may allow for further optimizations in the grounding process.

```

on(0, N) :- input(N), inpfix(N).
{ on(0, K) } :- not inpfix(K), layer(0, N), K = 1..N.

```

Encoding 13: Encoding of input region based on fixed bits using rule for fixed bits and choice for unfixed bits

3.2.4 Encoding of robustness

As already discussed in Section B.3, for the inference on a logic program, Clasp uses model enumeration. This means that it needs to enter every leaf node of a tree of partial solutions that is a model (answer set) of this logic program. On the other hand, if part of this tree does not contain any model, it may prune this whole branch.

The robustness as defined in the Definition 1.2.1 does not depend on inputs that yield output values for which the evaluation function h_p does yield 0. By the previous observation it is good to remove all such inputs from the models of the logic program.

Clingo itself can not directly compute the quantitative robustness of the binarised neural network. It can however still count the number of its models. By forbidding the desired output value in the logic program, only the partial solution that do not output this desired value (thus break the robustness) are left as models of the logic program. When using the evaluation function h that assigns 0 to forbidden output values and 1 to all other values and weight function $w(x) = 1$, the quantitative robustness can be easily computed from the number of found models and the size of the input region. The size of the input region can in turn be found out using Lemmas 1.2.6 and 1.2.7.

$$Q(I) = \frac{\sum_{i \in I} w(i) \cdot \overline{h_p(F(i))}}{\sum_{i \in I} w(i)} = \frac{\sum_{i \in I} 1 \cdot \overline{h_p(F(i))}}{\sum_{i \in I} 1}$$

As only the models of such logic program yield $\overline{h_p}(F(i)) = 1$, the upper part can be substituted for the number of models. The lower part corresponds to the size of the input region.

$$Q(I) = \frac{\#models}{||input_region||}$$

Forbidden outputs can be specified using constraints on literals with predicate symbol output. In case of an evaluation function $\overline{h_p}$ from Definition 1.2.3 that assigns 0 to output value k , the constraint would be:

```
:- output(k) .
```

as the value k does lead to the desired output and is thus forbidden. On the other hand, to encode t -target robustness (see end of Section 1.2.1) following constraint could be used:

```
:- not output(t) .
```

as only the output value t is not desired.

3.3 Encoding robustness of BNN into logic program

For transcription of the binarised neural network and the robustness problem, I use Python script. This script takes on input a binarised neural network in form of a directory of csv files, a base input in form of text file and possibly multiple other parameters (see Section 3.3.3). It then encodes this model into logic program using specified parameters. Finally, it executes the logic program using Clingo, counting all models with output that differs from the desired one.

3.3.1 Structure of saved BNN

The script allows for evaluation of binarised neural networks in the form of binarised multi-layer perceptron with batch normalization.

Binarised neural network is represented as a directory containing a subdirectory named `blkX` for every inner layer X of the network and a subdirectory named `out_blk` for the output Argmax layer. Each inner block subdirectory then contain files `bn_bias.csv`, `bn_mean.csv`, `bn_var.csv`, `bn_weight.csv`, `lin_bias.csv` and `lin_weight.csv`. Each

of these files contain vector or matrix corresponding to the parameter of the layer. The last Argmax layer subdirectory contains only files `lin_bias.csv` and `lin_weight.csv`.

Further follows specification of individual files.

`bn_bias.csv`

File contains a vector specified using a single line of comma separated values. This vector corresponds to the biases $\vec{\gamma}$ of a binarised single-layer perceptron with batch normalization, i -th entry corresponds to the bias of perceptron on i -th position of this layer.

`bn_mean.csv`

File contains a vector specified using a single line of comma separated values. This vector corresponds to the means $\vec{\mu}$ of a binarised single-layer perceptron with batch normalization, i -th entry corresponds to the mean of perceptron on i -th position of this layer.

`bn_var.csv`

File contains a vector specified using a single line of comma separated values. This vector corresponds to the variances $\vec{\sigma}^2$ of a binarised single-layer perceptron with batch normalization. To get the standard deviations $\vec{\sigma}$, piecewise square root must be applied to this vector. i -th entry corresponds to the variance of perceptron on i -th position of this layer.

`bn_weight.csv`

File contains a vector specified using a single line of comma separated values. This vector corresponds to the weights $\vec{\alpha}$ of a binarised single-layer perceptron with batch normalization, i -th entry corresponds to the weight of perceptron on i -th position of this layer.

`lin_bias.csv`

File contains a vector specified using a single line of comma separated values. This vector corresponds to the biases $\vec{b}$ of a binarised single-

layer perceptron with batch normalization, i -th entry corresponds to bias the perceptron on i -th position of this layer.

`lin_weight.csv`

File contains a matrix specified using a table of values with comma as separator of columns. This matrix corresponds to the weights $\mathbf{w}$ of a binarised single-layer perceptron with batch normalization. Values of this matrix are ± 1 -binarised. Vector constituted by the i -th row of the matrix $\mathbf{w}$ corresponds to the vector of weights from previous layer to the perceptron on i -th position of this layer. Entry on i -th row and j -th column corresponds to the weight from j -th position of the previous layer to i -th position of this layer.

3.3.2 Transformation of BNN

After loading, the binarised neural network with batch normalization (Definitions 1.1.5 and 1.1.7) is transformed into the whole number-valued form using Python package Numpy [29].

Further I show the whole transformation with representation using matrices and vectors contrary to vectors and scalar values that were used so far. When using vectors I assume they represented by a column of values. In the implementation, they are transposed when loading from files. To distinguish between them, I will use $\cdot$ for dot product, $\star$ for pointwise product and $\div$ for pointwise division.

Transformation of inner layer

Starting with binarised single-layer with batch normalization Definition 1.1.5:

$$t^{\text{B}}(\vec{x}) \equiv \vec{\alpha} \star \left(((\vec{b} + \mathbf{w} \cdot \vec{x}) - \vec{\mu}) \div \vec{\sigma} \right) + \vec{\gamma} \geq 0$$

Using Lemma 1.1.2, the expression can be altered to use only a single bias and a matrix of weights.

$$b'_i = \begin{cases} b_i - \mu_i + \frac{\sigma_i}{\alpha_i} \cdot \gamma_i & \frac{\alpha_i}{\sigma_i} > 0 \\ \gamma_i & \frac{\alpha_i}{\sigma_i} = 0 \\ -b_i + \mu_i - \frac{\sigma_i}{\alpha_i} \cdot \gamma_i & \frac{\alpha_i}{\sigma_i} < 0 \end{cases}$$

$$\vec{w}_i' = \begin{cases} \vec{w}_i & \frac{\alpha_i}{\sigma_i} > 0 \\ \vec{0} & \frac{\alpha_i}{\sigma_i} = 0 \\ -\vec{w}_i & \frac{\alpha_i}{\sigma_i} < 0 \end{cases}$$

$$t^{\mathbb{B}}(\vec{x}) \equiv \vec{b}' + \mathbf{w}' \cdot \vec{x} \geq 0$$

In the encoding of weight, there is now a possibility for some rows to be vectors of zeros. That is not a problem as they can still be encoded into the Clingo language. In aggregate expressions any whole numbers may be used.

To further simplify the encoding into logic program, the ± 1 -binarised input vector $\vec{x}$ may be mapped to vector $\vec{x}_b$ of values $\{1, 0\}$ (Section 3.2.1).

$$t^{\mathbb{B}}(\vec{x}) \equiv \vec{b}' + \mathbf{w}' \cdot (2\vec{x}_b - \vec{1}) \geq 0$$

$$\vec{b}'' = \frac{\vec{b}' - \mathbf{w}' \cdot \vec{1}}{2}$$

$$t^{\mathbb{B}}(\vec{x}) \equiv \vec{b}'' + \mathbf{w}' \cdot \vec{x}_b \geq 0$$

Finally to prepare the perceptron for the transcription to logic program, pointwise flooring function may be used on the bias $\vec{b}''$. If the use of $\{1, 0\}$ -binarised perceptron is not desired, $\vec{b}'$ and $\vec{x}$ can be used in place of $\vec{b}''$ and $\vec{x}_b$ in this step.

$$t^{\mathbb{B}}(\vec{x}) \equiv \left\lfloor \vec{b}'' \right\rfloor + \mathbf{w}' \cdot \vec{x}_b \geq 0$$

Transformation of Argmax layer

The transformation of Argmax layer is arguably simpler than the one of inner layers. It starts directly with inner potential and outputs vector with 1 only at the position of largest value.

$$t^{am}(\vec{x}) = \arg \max(\vec{b} + \mathbf{w} \cdot \vec{x})$$

Again as shown in Sections 3.2.1 and 3.2.2, the input vector $\vec{x}$ may be mapped to vector $\vec{x}_b$ of values $\{1, 0\}$ to simplify the encoding of logic program.

$$t^{am}(\vec{x}) = \arg \max(\vec{b} + \mathbf{w} \cdot (2\vec{x}_b - \vec{1}))$$

$$\vec{b}' = \frac{\vec{b} - \mathbf{w} \cdot \vec{1}}{2}$$

$$t^{am}(\vec{x}) = \arg \max(\vec{b}' + \mathbf{w} \cdot \vec{x}_b)$$

As shown in Lemma 3.1.2, the bias may be split to its integer part and fractional part and ordering made on fractional part. This ordering should have the position with highest fractional part as highest priority. The algorithm for the sorting needs to be stable as in the event of the fractional part being the same value, lower position has higher priority.

$$\vec{b}' = \lfloor \vec{b}' \rfloor + \{\vec{b}'\}$$

$$ord = \arg \text{sort}(-\{\vec{b}'\})$$

3.3.3 Evaluator framework

As a part of implementation, I have created python framework a for simple encoding and execution of the verification task. The framework resides in directory `/testing/verificator.py`. It allows for specification of a number of different parameters on command line.

To get all possible parameters of this framework, see:

```
./verificator.py -help
```

4 Evaluation

In this part I discuss the speed of individual encoding components. For the best combination of components. I also show the capabilities of implemented framework on multiple BNN models and inputs.

4.1 Methodology of evaluation

For the evaluation of my framework, I have used inputs from MNIST dataset [15], the same as were used for the evaluation of BNNQuanalyst [9]. In the next chapter I will be referring to instances of inputs corresponding to numbers 0 to 9 as I0 to I9.

Models for evaluation have been taken from [9]. The architectures of different models is written in Table 4.1. Values in the column *Architecture* correspond to layers sizes. First is size of the input vector, then follow inner layers and last is size of the output. Training of BNN models is not part of this thesis and thus is not discussed here.

Model	Architecture	Model	Architecture
M1	100:100:10	M7	100:50:20:10
M2	100:50:10	M8	16:25:20:10
M3	400:100:10	M9	36:15:10:10
M4	64:10:10	M10	16:64:32:20:10
M5	784:100:10	M11	25:25:25:20:10
M6	100:100:50:10	M12	784:50:50:50:50:10

Table 4.1: Architectures of models

Unless otherwise specified, the framework was evaluated on Dell G3 3579. For full specifications of both the machine and the system, see Appendix D.

4.1.1 Consistency of evaluation

To measure consistency of speed of the evaluator itself, first I have measured the time to compute robustness of model M7 and input

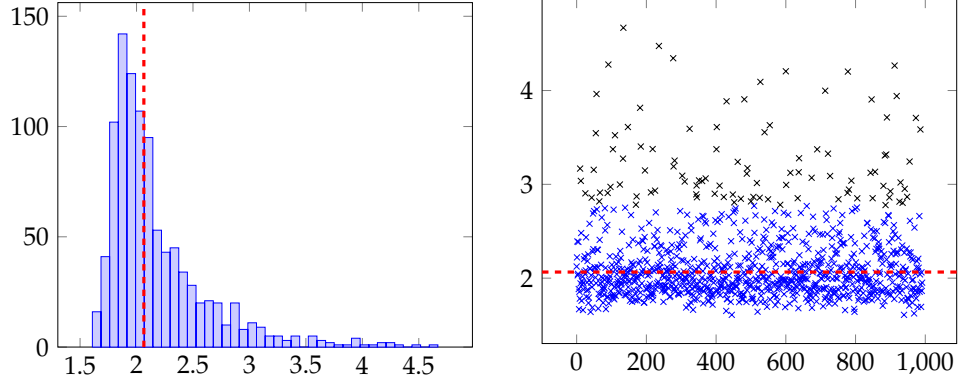


Figure 4.1: Scatter plot and histogram of short inputs.

Histogram on the left shows distribution of the computation time. Scatter plot on the right shows dependency of computation time on the order of computation. Red dotted lines are average of low 90 % lowest times (blue marks of graphs on the right).

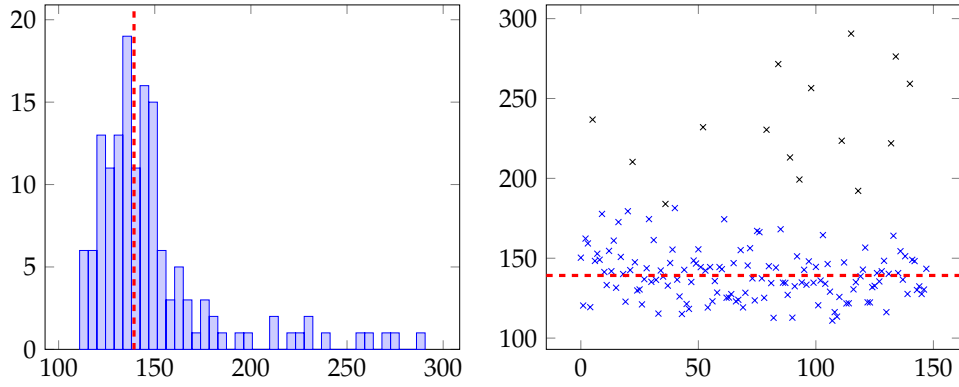


Figure 4.2: Scatter plot and histogram of long inputs.

Histogram on the left shows distribution of the computation time. Scatter plot on the right shows dependency of computation time on the order of computation. Red dotted lines are average of 90 % lowest times (blue marks of graphs on the right).

region $HD(I9, 2)$. I have evaluated this using 4 CPU cores. Resulting times of this evaluation are shown in Figure 4.1.

I have also measured time to compute robustness of model M11 and input region $HD(I0, 8)$ using 8 CPU cores. Resulting times of this evaluation are shown in Figure 4.2.

The plots of Figures 4.1 and 4.2 show that the time of computation is not consistent. It is however mostly consistent on 90 % lowest times. For this, I will always run the evaluation 4 times and take average of the 3 best evaluations.

4.2 Evaluation of encodings

Throughout this thesis I came up with multiple implementations of computation on perceptron, argmax layer and of encoding of fixed bits. In this section I provide comparison of these implementations using the methodology specified in Section 4.1.1.

Objective of this section is to demonstrate the size of difference between individual encodings. It stands as an argument for choosing an encoding for further analysis.

I have used following parameters unless specified otherwise:

- Perceptron encoding: Encoding 3
- Output layer encoding: Encoding 9
- Hamming encoding: Encoding 11
- Fixed bit encoding: last $n - k$ bits fixed
- Time limit: 300 s
- Parallel threads: 8

4.2.1 Evaluation of perceptron computation

For the perceptron, I have constructed 3 encodings, Encodings 1, 2 and 3.

4. EVALUATION

		M1		M2		M7	
		I0	I7	I0	I7	I0	I7
$d = 0$	Enc. 1	34.428	34.706	11.304	11.263	—	116.602
	Enc. 2	0.111	0.110	0.057	0.056	0.063	0.062
	Enc. 3	0.124	0.123	0.063	0.062	0.071	0.069
$d = 1$	Enc. 1	—	—	116.602	191.818	—	—
	Enc. 2	0.292	0.552	0.117	0.180	0.373	0.645
	Enc. 3	0.321	0.373	0.128	0.152	0.474	0.396
$d = 2$	Enc. 1	—	—	—	—	—	—
	Enc. 2	3.114	2.604	1.472	1.130	1.974	1.935
	Enc. 3	2.535	3.125	1.175	1.188	3.031	2.288
$d = 3$	Enc. 1	—	—	—	—	—	—
	Enc. 2	65.365	54.281	33.758	27.033	41.378	40.131
	Enc. 3	54.734	47.627	34.122	22.653	55.003	45.514

Table 4.2: Computation time for models M1, M2 and M7 on $HD(I0, d)$ and $HD(I7, d)$ for $d \in \{0, 1, 2, 3\}$ and different implementations of perceptron. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Symbols — in the table mean that less than 3 of 4 evaluations finished before timeout. Timeout was set to 300 s.

Over hamming distance

I have measured the time to evaluate robustness over input regions based of hamming distance $HD(I0, d)$ and $HD(I7, d)$ for $d \in \{0, 1, 2, 3\}$ on models M1, M2 and M7. Results are in Table 4.2.

There is a large difference between the time of evaluation logic programs storing value of inner potential (Encoding 1) to those which are using the value of inner potential directly (Encodings 2 and 3). As already discussed in Section 3.2.1, this is due to the encoding resulting in large ground program and thus inherently slower evaluation.

Difference between Encoding 2 using $(+1, -1)$ -binarization of inputs and Encoding 3 using $(0, 1)$ -binarization is miniscule.

4. EVALUATION

		M1		M2		M7	
		I0	I7	I0	I7	I0	I7
$F = 0$	Enc. 1	8.765	7.366	3.723	3.239	—	123.503
	Enc. 2	0.117	0.116	0.060	0.059	SegFault	0.065
	Enc. 3	0.105	0.104	0.054	0.053	0.059	0.058
$F = 8$	Enc. 1	33.032	25.277	9.153	7.932	—	—
	Enc. 2	0.150	0.136	0.070	0.073	SegFault	SegFault
	Enc. 3	0.120	0.119	0.063	0.069	0.093	0.098
$F = 16$	Enc. 1	—	—	256.537	—	—	—
	Enc. 2	1.910	1.594	0.762	0.853	1.850	SegFault
	Enc. 3	1.318	1.661	0.482	0.775	1.534	1.654
$F = 22$	Enc. 1	—	—	—	—	—	—
	Enc. 2	152.552	—	63.089	65.880	124.291	126.506
	Enc. 3	102.996	—	50.311	71.003	101.660	111.104

Table 4.3: Computation time for models M1, M2 and M7 on $R(I0, \{F, \dots, 100\})$ and $HD(I7, \{F, \dots, 100\})$ for $F \in \{0, 8, 16, 22\}$ and different implementations of perceptron.

Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Symbols — in the table mean that less than 3 of 4 evaluations finished before timeout. Timeout was set to 300 s. Evaluations with “SegFault” in table ended with segmentation fault. This error was caused by solver Clasp.

Over fixed bits

I have measured the time to evaluate robustness of models M1, M2 and M7 over input regions based on fixed bits $R(I0, \{F + 1, \dots, 100\})$, $R(I7, \{F + 1, \dots, 100\})$, where $F \in \{0, 8, 16, 22\}$, that is 0, 8, 16 and 22 free bits at the begining of the input vector. Results are in Table 4.3.

Similiarly to evaluation over hamming distance, there are large differences between Encoding 1 and Encodings 2 and 3. Additionally, Encoding 3 slightly outperforms Encoding 2.

4.2.2 Evaluation of argmax computation

For the argmax layer, I have constructed 6 encodings, Encodings 4, 5, 6, 7, 8 and 9.

		M1		M2		M7	
		I0	I7	I0	I7	I0	I7
$d = 0$	Enc. 4	3.216	3.126	0.758	0.779	0.201	0.194
	Enc. 5	3.226	3.197	0.828	0.804	0.185	0.176
	Enc. 6	0.766	0.749	0.202	0.196	0.087	0.082
	Enc. 7	0.857	0.864	0.235	0.229	0.095	0.091
	Enc. 8	0.112	0.111	0.057	0.057	0.065	0.062
	Enc. 9	0.111	0.110	0.056	0.056	0.064	0.062
$d = 1$	Enc. 4	5.050	7.751	0.965	1.510	0.556	0.982
	Enc. 5	5.005	5.977	1.115	1.303	0.548	0.833
	Enc. 6	1.463	2.064	0.340	0.452	0.419	0.738
	Enc. 7	1.568	2.164	0.369	0.495	0.475	0.724
	Enc. 8	0.294	0.549	0.117	0.181	0.375	0.619
	Enc. 9	0.294	0.544	0.117	0.184	0.344	0.634
$d = 2$	Enc. 4	51.059	18.823	12.264	8.789	3.285	2.899
	Enc. 5	18.904	16.342	5.806	4.794	2.436	2.308
	Enc. 6	11.547	7.927	3.873	2.713	2.499	2.151
	Enc. 7	11.007	7.408	3.712	2.931	2.186	2.068
	Enc. 8	2.991	2.640	1.380	1.117	2.030	2.044
	Enc. 9	3.346	2.659	1.390	1.065	2.259	1.755
$d = 3$	Enc. 4	—	—	199.465	133.432	75.185	74.186
	Enc. 5	—	—	163.086	104.734	57.775	51.720
	Enc. 6	191.495	184.556	118.226	80.181	47.022	50.710
	Enc. 7	185.261	175.619	110.676	79.620	47.209	45.417
	Enc. 8	73.132	53.837	33.121	27.664	45.230	41.119
	Enc. 9	70.076	55.665	34.035	25.567	43.001	40.295

Table 4.4: Computation time for models M1, M2 and M7 on $HD(I0, d)$ and $HD(I7, d)$ for $d \in \{0, 1, 2, 3\}$ and different implementations of argmax layer. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Symbols — in the table mean that less than 3 of 4 evaluations finished before timeout. Timeout was set to 300 s.

Over hamming distance

I have measured the time to evaluate robustness over input regions based of hamming distance $HD(I0, d)$ and $HD(I7, d)$ for $d \in \{0, 1, 2, 3\}$ on models M1, M2 and M7. Results are in Table 4.4.

Encodings may be split into 3 groups by the computation time. Encodings 4 and 5, Encodings 6 and 7 and Encodings 8 and 9.

Encodings 4 and 5 result in a slow grounded program. These encodings use predicate `symbol potential`, with inner potential computed by sum aggregate over both positive and negative literals of `symbol` on according to the default negation. Grounder Gringo does not see that either a positive or negative version of literal must be true at a time. The grounded program contains for each perceptron atoms of `symbol potential` with values $b - m$ to $b + m$ where b is bias of the perceptron and m length of input of the perceptron, while only values that give the same remainder after division by 2 as $b + m$ are feasible.

In Encoding 4, atoms of `symbol potential` are then used in max aggregate. When grounding this type of aggregate, Gringo generally orders its terms and for each of them adds a rule that asserts the aggregate to be equal to the largest term of those whose condition is consistent with the answer set. As there are many different atoms of `symbol potential`, this then results in a large set of rules.

In case of Encoding 5, the part dependent on atoms of `symbol potential` is the constraint on output. The constraint ground into constant number of rules for each pair of potential atoms of different outputs.

Moving to Encoding 6, the number of atoms of `symbol potential` halves. This results in quartering of the number of rules and a significant speedup. Encoding 7 is similar. Resulting logic program still has atoms for individual values of inner potential, however now it has them for each pair of distinct outputs individually.

Finally, Encodings 8 and 9 hold only a single constraint based of aggregate for every pair of outputs. This largely decreases the size of the grounded program as all but these have been storing some form of all possible inner potential values. These two are grounding into a rule directly computing the inner potential.

4. EVALUATION

		M1		M2		M7	
		I0	I7	I0	I7	I0	I7
$F = 0$	Enc. 4	3.147	3.030	0.752	0.762	0.193	0.183
	Enc. 5	3.012	2.913	0.785	0.760	0.173	0.164
	Enc. 6	0.689	0.667	0.183	0.177	0.082	0.076
	Enc. 7	0.787	0.766	0.214	0.209	0.091	0.085
	Enc. 8	0.106	0.105	0.054	0.054	0.060	0.058
	Enc. 9	0.105	0.104	0.054	0.053	0.060	0.058
$F = 8$	Enc. 4	3.928	4.220	0.932	0.931	0.332	0.302
	Enc. 5	11.557	12.393	1.772	1.777	0.256	0.246
	Enc. 6	1.320	1.399	0.281	0.288	0.130	0.139
	Enc. 7	1.422	1.502	0.316	0.326	0.136	0.173
	Enc. 8	0.120	0.119	0.063	0.070	0.118	0.114
	Enc. 9	0.119	0.118	0.062	0.066	0.108	0.095
$F = 16$	Enc. 4	29.282	30.253	5.244	6.693	4.212	4.926
	Enc. 5	30.858	26.547	7.852	6.070	3.497	3.960
	Enc. 6	9.125	7.092	2.766	2.151	1.895	2.125
	Enc. 7	8.877	7.003	3.022	2.305	1.947	2.224
	Enc. 8	1.316	1.589	0.464	0.746	1.627	1.663
	Enc. 9	1.321	1.560	0.477	0.714	1.533	1.633

Table 4.5: Computation time for models M1, M2 and M7 on $R(I0, \{F, \dots, 100\})$ and $R(I7, \{F, \dots, 100\})$ for $F \in \{0, 8, 16\}$ and different implementations of perceptron. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4).

Over fixed bits

I have measured the time to evaluate robustness of models M1, M2 and M7 over input regions based on fixed bits $R(I0, \{F + 1, \dots, 100\})$, $R(I7, \{F + 1, \dots, 100\})$, where $F \in \{0, 8, 16\}$, that is 0, 8 and 16 free bits at the beginning of the input vector. Results are in Table 4.5.

As in the Table 4.4, the encodings were separated to 3–4 groups by the computation time. The differences are similliar. In this evaluation, Encoding 9 is marginally better than Encoding 9.

		M4		M5		M7	
		I0	I7	I0	I7	I0	I7
$F = 0$	Enc. 11	0.010	0.010	0.628	0.626	0.080	0.077
	Enc. 12	0.010	0.010	0.605	0.607	0.093	0.075
	Enc. 13	0.008	0.008	0.487	0.483	0.042	0.042
$F = 8$	Enc. 11	0.013	0.013	0.637	0.637	0.114	0.101
	Enc. 12	0.012	0.012	0.609	0.610	0.098	0.096
	Enc. 13	0.010	0.011	0.495	0.488	0.065	0.079
$F = 16$	Enc. 11	0.172	0.243	0.658	0.713	1.519	1.624
	Enc. 12	0.146	0.208	0.617	0.636	1.241	1.466
	Enc. 13	0.137	0.204	0.503	0.520	1.157	1.366
$F = 20$	Enc. 11	2.845	4.575	0.701	1.310	24.801	27.936
	Enc. 12	2.807	3.818	0.635	0.867	18.880	23.989
	Enc. 13	2.772	3.505	0.520	0.740	18.500	24.245

Table 4.6: Computation time for models M1, M2 and M7 on $R(I0, \{F, \dots, n\})$ and $R(I7, \{F, \dots, n\})$ where n is size of input of model and $F \in \{0, 8, 16, 20\}$, for different implementations of input region based on fixed bits. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4).

4.2.3 Evaluation of fixed bits encoding

In the implementation part of this thesis, I have introduced 3 possible encodings of input region based on fixed bits, Encodings 11, 12 and 13. I have measured computation time of verification task using these encodings on models M4, M5 and M7 on input regions $R(I0, \{F+1, \dots, n\})$ and $R(I0, \{F+1, \dots, n\})$ where n is the input length of used model, for $F \in \{0, 8, 16, 20\}$. Results are in Table 4.6.

By a moderate margin, Encoding 13 seems to be the fastest, especially in cases with a large input vector (M5). In cases of the size of input vector being small (M4), the margins are smaller.

4.3 Evaluation of performance of the implemented framework

In the last section, I have compared the performance of my framework over different encodings of layers and constraints on the input. In this

section I proceed to evaluate 12 models of BNN classifiers trained on the MNIST dataset.

For the evaluation I have used following parameters:

- Perceptron encoding: Encoding 3
- Output layer encoding: Encoding 9
- Hamming encoding: Encoding 13
- Fixed bit encoding: last $n - k$ bits fixed
- Base input instance: Instance 0
- Time limit: 600 s
- Parallel threads: 8

4.3.1 Over hamming distance

Using my framework, I have evaluated robustness of all models over input region based on the hamming distance of up to 4. The results of this evaluation are shown in Figure 4.3, values of the individual measurement and of the computed robustness for these models are also in Table E.1. From the Figure 4.3 it seems that the framework has time complexity close to linear to the size of the input region. Due to the limitations of Clingo given by the inference algorithm based on enumeration of solutions, the time complexity can not be lower than linear to the number of solutions (the number of inputs with differing outputs).

4.3.2 Over fixed bits

Using my framework, I have evaluated robustness of all models over input regions based on fixed bits with up to 24 free bits. The results of this evaluation are shown in Figure 4.4, values of the individual measurement and of the computed robustness for these models are also in Table E.2. The time complexity seems to be similar to the one of evaluation using input region based on hamming distance. The data does not suggest evaluation over any type of input region being significantly faster than the other.

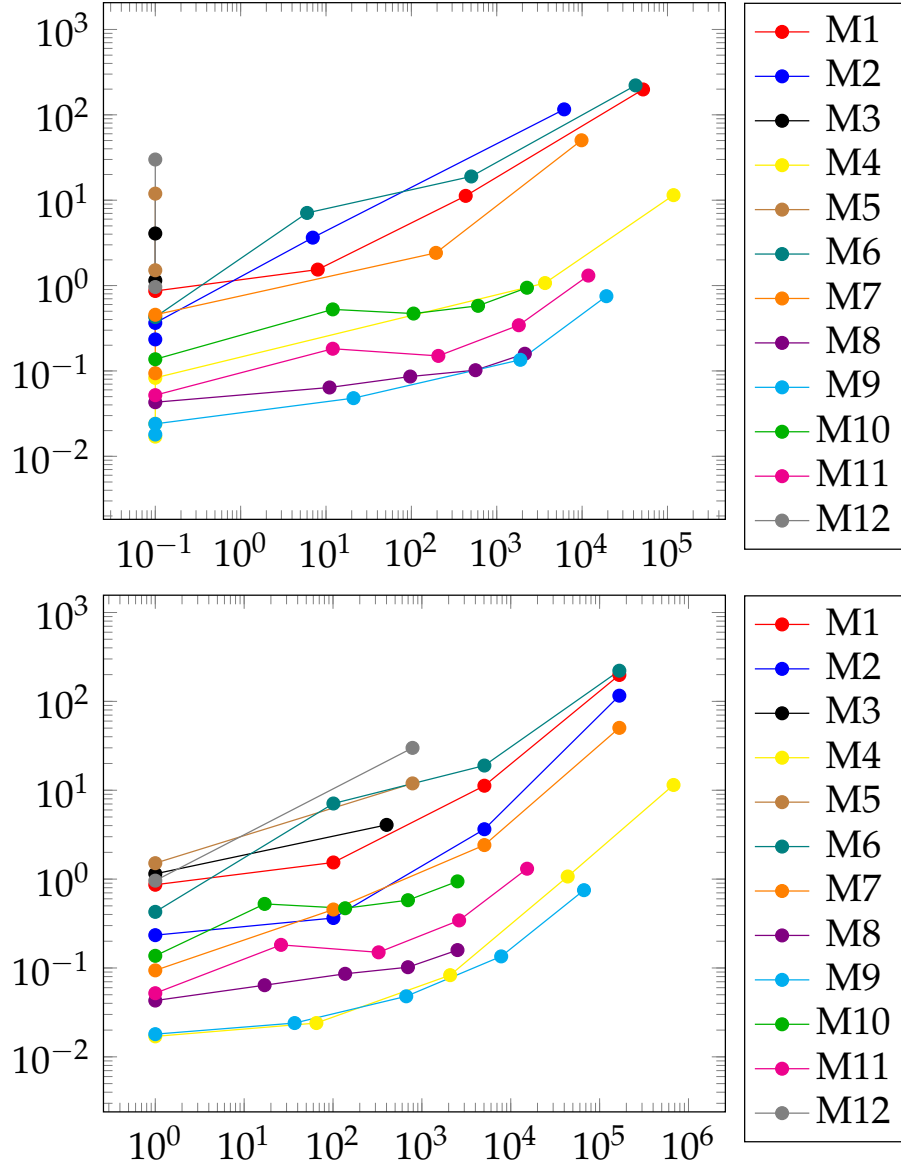


Figure 4.3: Computation time for verification of all models on $HD(I_0, d)$ for $d \in \{0, 1, 2, 3, 4\}$. Values in the figure were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Evaluations that did not finish before timeout are not shown in this figure. Timeout was set to 600 s.

The upper figure shows dependency of computation time on the number of found models (inputs with output different from that of base input), the bottom figure shows dependency of computation time on the size of input region. In case of Data for these figures are in Table E.1.

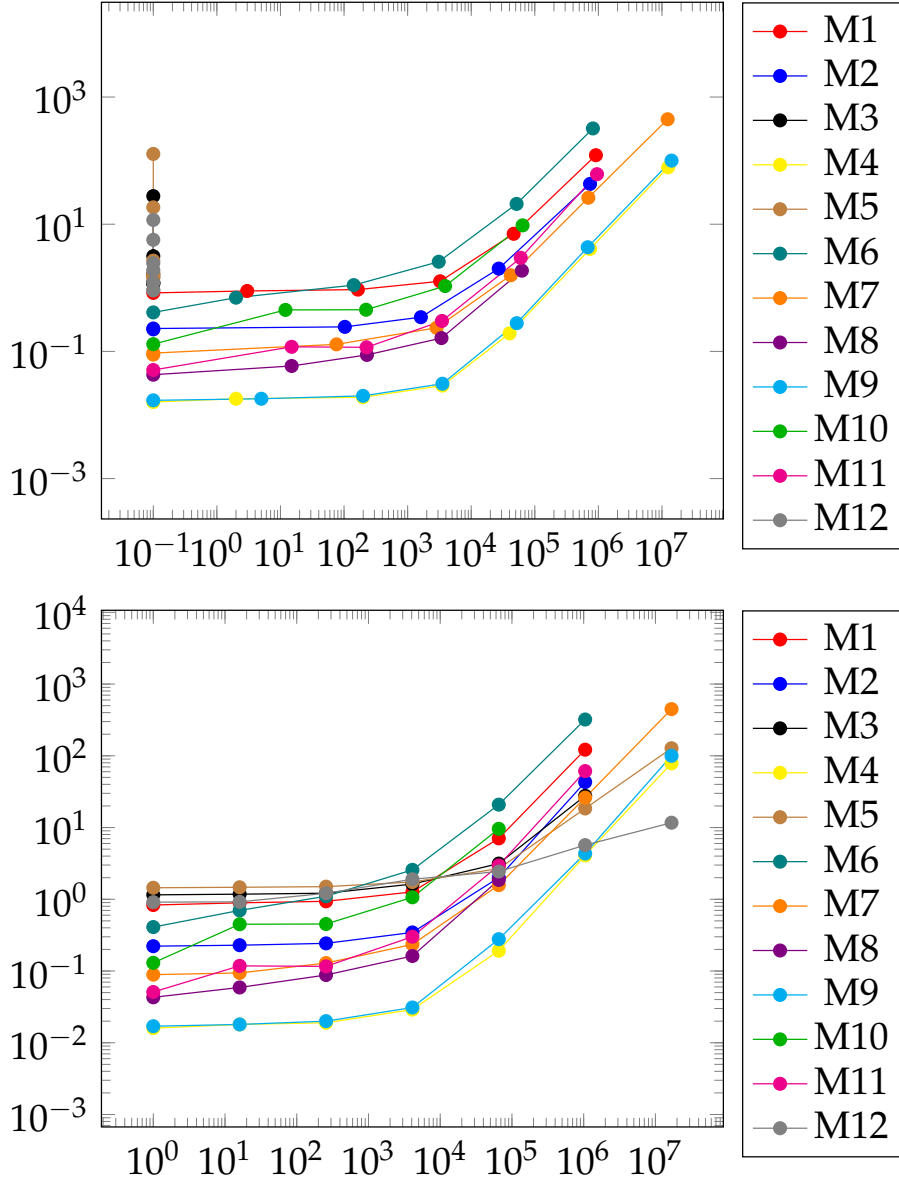


Figure 4.4: Computation time for verification of all models on $R(I0, \{F + 1, \dots, n\})$ where n is size of input of model and $F \in \{0, 4, 8, 12, 16, 20, 24\}$. Values in the figure were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Evaluations that did not finish before timeout are not shown in this figure. Timeout was set to 600 s.

The upper figure shows dependency of computation time on the number of found models (inputs with output different from that of base input), the bottom figure shows dependency of computation time on the size of input region. Data for these figures are in Table E.2.

Conclusion

In this thesis, I have provided definition of binarised neural networks and shown that even with the batch normalization, which comes from training of these networks, they can be encoded into operations only using integers.

Further, I have defined quantitative robustness of arbitral function as an integral or sum of weighted evaluation of input over input region. Further, I have defined qualitative robustness using the previously defined quantitative robustness. For both of these definitions I have shown and proven multiple properties, such as bounds of quantitative robustness and independence of qualitative robustness on weight function. Finally, I have shown definition of input region based on the Hamming distance and fixed bits. For both of these regions I have derived a direct way of computing their size.

The third chapter focussed on Answer set programming, which is a branch of logic programming I have used in the implementation part of this thesis. I provide definition of a logic program and its semantics using the reduct of program. All of this is illustrated on examples. For a demonstration of a solver of answer set programming paradigm, I refer to appendix, where I show syntax of Clingo, consisting of grounder Gringo and solver Clasp. In this appendix I also point out some problems that may lead to large grounded program and high solving time.

In the next chapter I have created an encoding of binarised neural network robustness problem into a logic program. This encoding consists of a variable part containing parameters computed from the binarised neural network and a constant part providing the computation over this network. To create the variable part, I use Python.

Finally, the last part of this thesis is an evaluation of the encoding. The metric of evaluation is computational time. I compare this time between different encodings of the constant part. Using the best found encoding, I have evaluated robustness of 12 binarised neural networks trained on MNIST dataset over input region based on the Hamming distance and on fixed bits. My implementation proves to be able to evaluate binarised neural network on an input region with size up to 10^7 in 600 s on a regular computer.

A Proofs and examples

A.1 Proofs

Lemma A.1.1. *The Dirichlet function D is robust on any interval $\langle k, l \rangle$ where k, l are rational numbers, $k \neq l$.*

Proof. As shown in the Example 3.1.1 of [30], the Dirichlet function has Lebesgue integral on interval $[0, 1]$ with a value equal to 0.

Lets prove that for every rational number $a \in \mathbb{Q}$ and real number $b \in \mathbb{R}$ following statements are equivalent:

1. b is rational
2. $a + b$ is rational

1. $\implies$ 2.:

Since both a and b are rational, by definition they can be written as a fraction of integers

$$a = \frac{p}{q}, b = \frac{p'}{q'}$$

The sum can be expressed as a fraction of integers, thus is also rational.

$$a + b = \frac{pq' + p'q}{qq'}$$

2. $\implies$ 1.:

The b can be written using the rational a and the sum of a and b .

$$b = (a + b) + (-a)$$

Now since both $(-a)$ and $(a + b)$ are rational, b was already proven to be rational in the opposite side implication.

Lets show that the integral of Dirichlet function D is equal to 0 on any interval bounded by rational numbers. The limits of the definite integral can be transformed by removing k and adding it to the argument of D . As was proven previously, for rational number k , $D(x + k) = D(x)$.

$$\int_k^l D(x)dx = \int_0^{l-k} D(x+k)dx = \int_0^{l-k} D(x)dx$$

As the dirichlet function is nonnegative, following is true.

$$0 \leq \int_0^{l-k} D(x)dx \leq \int_0^{\lceil l-k \rceil} D(x)dx$$

The right-hand side integral can be split into unit-long parts.

$$\int_0^{\lceil l-k \rceil} D(x)dx = \int_0^1 D(x)dx + \int_1^2 D(x)dx + \dots + \int_{\lceil l-k \rceil - 1}^{\lceil l-k \rceil} D(x)dx$$

Finally, as every unit integral has rational bounds, it can be transformed to integral with 0 as lower bound like already shown.

$$\int_u^{u+1} D(x)dx = \int_0^1 D(x+u)dx = \int_0^1 D(x)dx = 0$$

$$0 \leq \int_k^l D(x)dx \leq \int_0^{\lceil l-k \rceil} D(x)dx = 0$$

To show the robustness of Dirichlet function, the quantitative robustness with respect to constant weight function w_1 and identity as the evaluation function can be used.

$$Q_{w_1, id}(\langle k, l \rangle) = \frac{\int_k^l 1 \cdot D(x)dx}{\int_k^l 1} = \frac{0}{l-k} = 0$$

□

Lemma A.1.2. *Incremental building of the answer set of the basic logic program. Let (Σ, Π) be a basic logic program with at most one literal in the head of every rule. Let*

$$S_0 = \emptyset,$$

$$S_{i+1} = S_i \cup \{\varphi \mid \{\varphi\} = \text{head}(r), \text{body}^+(r) \subseteq S_i\} \text{ for each } i \geq 0.$$

Denote by S_∞ the union of all S_i for $i \geq 0$. If it holds that:

- for every Σ -atom φ , S_∞ contains at most one of literals φ , $\neg\varphi$, and
- S_∞ is consistent with every constraint of Π ,

then S_∞ is the only answer set of program (Σ, Π) . Else (Σ, Π) has no answer set.

For a proof of this lemma, see Lemma A.1.2.

Proof. S_∞ does contain only (some of) heads of rules from Π , each of these being Σ -literals. If S_∞ does also not contain both φ and $\neg\varphi$ for any Σ -atom φ , S_∞ is a partial interpretation of Σ .

By definition S_∞ contains the literal from the head of each rule, that has its body consistent with the S_∞ . For this the S_∞ is consistent with every non-constraint rule of the logic program.

Let $rank : S_\infty \rightarrow \mathbb{N}_0$ be a function that assigns to every literal $\varphi \in S_\infty$ the lowest value n such that $\varphi \in S_n$. Let there be some answer set A of (Σ, Π) such that $S_\infty \setminus A = \Delta \neq \emptyset$. Let α be a literal from Δ with the lowest $rank$. By definition, $rank(\alpha) \geq 1$ as S_0 is empty. As the literal $\alpha \in S_{rank(\alpha)}$ and $\alpha \notin S_{rank(\alpha)-1}$, either α is in a head of some fact (thus $rank(\alpha) = 1$ and α needs to be in A for it to be consistent), or there must be some rule $r \in \Pi$ such that $body^+(r)$ is not consistent with $S_{rank(\alpha)-2}$ and is consistent with $S_{rank(\alpha)-1}$. (The rule r is what made the literal α in $S_{rank(\alpha)}$.) But A does contain all the literals from $S_{rank(\alpha)-1}$. (Literal α has the lowest rank of all literals from S_∞ not included in A .) Thus A is not consistent with r and is not an answer set of (Σ, Π) . By contradiction, for every answer set B of (Σ, Π) , $S_\infty \subseteq B$. As the answer set is by definition minimal, the S_∞ is the only possible answer set of (Σ, Π) . $\square$

A.2 Examples

Example A.2.1. The ground instantiation of the logic program (Σ_1, Π_1) defined in the Example 2.1.2 can be found as follows:

The fact $p(True)$. does not contain any variable nor function symbol. For this, it is ground. The same holds for the fact $q(5, True)$. and the rule $p(False) \leftarrow \text{not } p(True)$.

To ground the rule $q(Y, True) \leftarrow q(X, (Y < X))$., this rule should be first substituted for each rule with pair (X, Y) substituted for every element from $\sigma_{\mathbb{N}} \times \sigma_{\mathbb{N}}$, that is every element from $\mathbb{N}_0 \times \mathbb{N}_0$. Then in the second parameter of atom in the body, the expression substituted for $(Y < X)$ would be replaced with *True* or *False*, depending on the substitution for X and Y . This would lead to the ground instantiation $gr(\Sigma_1, \Pi_1)$ being infinite.

To fight this, we can allow only the instantiation of rule r in which only the substitutions of variables in the $body^+(r)$ leading to the

already existing atoms are allowed. After the stable model will be defined, it will be easy to see that this constraint does not remove any stable models of program, as every atom in it has to also be in the head of some rule.

In this example, we already know that the atom $p(5, \text{True})$ does exist. This means we can substitute $X \equiv 5$ and $(Y < X) \equiv \text{True}$. Also as $(Y < X) \equiv (Y < 5) \equiv \text{True}$, to successfully instantiate the rule, it has to hold that $Y < 5$. There are no other constraints on the rule, thus the rule instantiates into rules:

$$\begin{aligned} Y = 0 & : q(0, \text{True}) \leftarrow q(5, \text{True}). \\ Y = 1 & : q(1, \text{True}) \leftarrow q(5, \text{True}). \\ Y = 2 & : q(2, \text{True}) \leftarrow q(5, \text{True}). \\ Y = 3 & : q(3, \text{True}) \leftarrow q(5, \text{True}). \\ Y = 4 & : q(4, \text{True}) \leftarrow q(5, \text{True}). \end{aligned}$$

But now we have introduced new atoms on predicate q . We have to also add all instances of the rule, in which these atoms are allowed. As the individual atoms only differ in the first term, the instances are seen easily. The instances added by the new rules also do not introduce new atoms on predicate q , the ground instance is thus in this case finite.

$$\begin{aligned} X = 4, Y = 0 & : q(0, \text{True}) \leftarrow q(4, \text{True}). \\ X = 4, Y = 1 & : q(1, \text{True}) \leftarrow q(4, \text{True}). \\ X = 4, Y = 2 & : q(2, \text{True}) \leftarrow q(4, \text{True}). \\ X = 4, Y = 3 & : q(3, \text{True}) \leftarrow q(4, \text{True}). \\ X = 3, Y = 0 & : q(0, \text{True}) \leftarrow q(3, \text{True}). \\ X = 3, Y = 1 & : q(1, \text{True}) \leftarrow q(3, \text{True}). \\ X = 3, Y = 2 & : q(2, \text{True}) \leftarrow q(3, \text{True}). \\ X = 2, Y = 0 & : q(0, \text{True}) \leftarrow q(2, \text{True}). \\ X = 2, Y = 1 & : q(1, \text{True}) \leftarrow q(2, \text{True}). \\ X = 1, Y = 0 & : q(0, \text{True}) \leftarrow q(1, \text{True}). \end{aligned}$$

The full ground instantiation $gr((\Sigma_1, \Pi_1))$ is thus the program:

$$gr((\Sigma_1, \Pi_1)) = \left\{ \begin{array}{l} p(True). \\ q(5, True). \\ p(False) \leftarrow \text{not } p(True). \\ q(0, True) \leftarrow q(5, True). \\ q(1, True) \leftarrow q(5, True). \\ q(2, True) \leftarrow q(5, True). \\ q(3, True) \leftarrow q(5, True). \\ q(4, True) \leftarrow q(5, True). \\ q(0, True) \leftarrow q(4, True). \\ q(1, True) \leftarrow q(4, True). \\ q(2, True) \leftarrow q(4, True). \\ q(3, True) \leftarrow q(4, True). \\ q(0, True) \leftarrow q(3, True). \\ q(1, True) \leftarrow q(3, True). \\ q(2, True) \leftarrow q(3, True). \\ q(0, True) \leftarrow q(2, True). \\ q(1, True) \leftarrow q(2, True). \\ q(0, True) \leftarrow q(1, True). \end{array} \right.$$

Example A.2.1 well illustrates the need for the logic programs to be well written. As was shown in the example, due to a single flawed rule, the size of the ground instantiation of the logic program is quadratic given the highest number in the first parameter of atoms of the predicate q .

B Clingo

Clingo is an integrated ASP system, consisting of a grounder Gringo and solver Clasp [27]. In this appendix I show basics of the Clingo language, Gringo as translation from Clingo to Aspif language and solving such program with Clasp.

B.1 Clingo language

Clingo language [13] is used for transcribing ASP programs in the Clingo system. Clingo language does allow for facts, rules and constraints just like the logic programs defined in this thesis. Clingo language is in fact almost identical to the already provided notation.

Example B.1.1. Transcription of a logic program from the Example 2.1.3 to Clingo language. On the left side is a set of rules from this example, on the right is equivalent program in the Clingo language.

$$\Pi = \left\{ \begin{array}{ll} p(a). & p(a). \\ p(b) \leftarrow p(a). & p(b) :- p(a). \\ p(c) \leftarrow p(b). & p(c) :- p(b). \\ p(x). & p(x). \\ p(y) \leftarrow p(x), p(a). & p(y) :- p(x), p(a). \\ p(z) \leftarrow p(x), p(c). & p(z) :- p(x), p(c). \\ p(n) \leftarrow p(a), p(k). & p(n) :- p(a), p(k). \end{array} \right.$$

B.1.1 Basic syntax of Clingo language

Unless specified otherwise, source of information in this section is [13].

Terms in the Clingo language are of multiple sorts. The simplest sort of terms are constants and strings. Constants match the regular expression `_*[a-z][A-Za-z0-9_']*`. They start with underscores followed by lowercase letter and a sequence of characters. Similarly, strings match the regular expression `"([^\\"↔]|\\["n"])*"`. They allow for any reasonable text.

Another sort of Clingo language signature are integers. Constant integers can be captured by the regular expression $-?(0|[1-9][0-9]*)$. As can be seen, zero character can only be at the start of zero number. Number zero (0) and negative zero (-0) are the same constants. The range of integers is platform dependent, they are either 32-bit or 64-bit.

Grounder Gringo also built-in support for arithmetic functions over integers. Following symbols are used: + (addition), - (subtraction), * (multiplication), / (integer division, rounding down), \ (modulo), ** (exponentiation), |...| (absolute value), & (bitwise AND), ? (bitwise OR), ^ (bitwise XOR), and ~ (bitwise complement). Arithmetic functions use infix notation.

Gringo also allows for comparison predicates over all terms. These are predicates that are evaluated during grounding. The comparison predicates are = (equal), != (not equal), < (less than), <= (less than or equal), > (greater than) and >= (greater than or equal). Integers are compared in the usual way, constants and strings are ordered lexicographically. All integers are smaller than constants which are smaller than strings. Additionally two special constants #sup and #inf do exist, constituting to the greatest and lowest term.

Another sorts of terms are functions and tuples. Such terms take form of $f(t_1, \dots, t_n)$ for the functions and $(t_1, \dots, t_n)$ for the tuples where f is a function symbol and $t_1, \dots, t_n$ are terms. The tuple has to have at least a single term long, in that case it is written as $(t_1,)$ to differentiate it from brackets. Function symbols are constrained in the same way as names of constants. Function with no terms as arguments is by the grounder seen as a constant with the function symbol as a name. Functions and tuples as terms do not have any interpretation. There is no mapping from function terms to some other terms as the name might suggest.

Finally, the last type of terms are variables. Variables can be matched with a regular expression $_*[A-Z][A-Za-z0-9_']*$. Variables are used to generalize rules of the logic program, a ground (variable-free) program is constructed during the process of grounding. A special variable $_$ is called an anonymous variable or wildcard. At the start of the grounding process, a fresh (not used elsewhere) variable term is substituted for each anonymous variable. It stands for a placeholder, meaning any term can take its place.

An atom is syntactically consistent with a constant or a function. The predicate symbol starts with any number of underscores `_` followed by a lowercase letter and any number of alphanumeric characters, underscores and `'`. Atom using predicate symbol with arity of length zero can be written without the parentheses. One predicate symbol can be used with different arities.

A literal is an atom or its classical negation. In the Clingo language, classical negation is indicated by minus symbol `-` before the literal.

Finally, each ASP rule r of form

$$l_0 \text{ or } \dots \text{ or } l_k \leftarrow l_{k+1}, \dots, l_m, \text{ not } l_{m+1}, \dots, \text{ not } l_n.$$

is transcribed into following expression of Clingo language:

$$l_0; \dots; l_k :- l_{k+1}, \dots, l_m, \text{ not } l_{m+1}, \dots, \text{ not } l_n.$$

Specially, if $\text{head}(r)$ is nonempty and $\text{body}(r)$ is empty, a shorthand transcription can be used:

$$l_0; \dots; l_k.$$

In the basic notation of rules, semicolons `;` and commas `,` can be freely interchanged. While this is true, I will try to stick to semicolons in heads and commas in bodies of rules for better readability. Where this is not possible (see Conditional literals), I will always point it out.

Example B.1.2. Transcription of terms and atoms of Example 2.1.1.

In the example, signature Σ_1 had sorts

$$\sigma_{\mathbb{B}} = \{True, False\}, \sigma_{\mathbb{N}} = \mathbb{N}_0.$$

On these sorts, function symbols $\vee, \wedge$ of arity $\sigma_{\mathbb{B}} \times \sigma_{\mathbb{B}} \rightarrow \sigma_{\mathbb{B}}$, and function symbols $=, <$ of arity $\sigma_{\mathbb{N}} \times \sigma_{\mathbb{N}} \rightarrow \sigma_{\mathbb{B}}$ were defined. Also, there were two predicate symbols, p of arity $\sigma_{\mathbb{B}}$, and q of arity $\sigma_{\mathbb{N}} \times \sigma_{\mathbb{B}}$. Finally, there were 3 variables X, Y, Z of sort $\sigma_{\mathbb{N}}$.

Let the interpretation of these function symbols be the usual one, that is $\vee, \wedge$ are logical OR and AND and $=, <$ are *equal* and *less than*. In Clingo language, there is no such thing as boolean values. However, we can still use integers 0 and 1 in their place. Function symbols $\vee$ and $\wedge$ can then be modelled by using bit and and bit or arithmetic functions of the Clingo language.

Function symbols $=$ and $<$ are a bit more problematic. In the Clingo language as defined so far there are no user defined function¹. When working with rules with these function symbols, we can however use comparison predicates to build multiple rules, each with its own assignment to the term value.

In the following, on the left side are Σ -terms of sort $\sigma_{\mathbb{B}}$, on the right side are their transcriptions into the Clingo language according to the above text. When the expression would need to be rewritten into multiple rules, different cases are inside curly brackets split by comma, for each case the substituted term is before the square brackets and added comparison predicates are inside the square brackets.

$True, False,$	$1, 0,$
$True \vee False, False \wedge True,$	$1 \text{ ? } 0, 0 \text{ \& } 1,$
$(42 = 42),$	$\{1 \text{ [} 42 = 42 \text{]},$
	$0 \text{ [} 42 \text{ != } 42 \text{]}\},$
$(X = 17),$	$\{1 \text{ [} X = 17 \text{]},$
	$0 \text{ [} X \text{ != } 17 \text{]}\},$
$(3 < Y) \vee (6 = 6),$	$\{1 \text{ ? } 1 \text{ [} 3 < Y, 6 = 6 \text{]},$
	$1 \text{ ? } 0 \text{ [} 3 < Y, 6 \text{ != } 6 \text{]},$
	$0 \text{ ? } 1 \text{ [} 3 \text{ >= } Y, 6 = 6 \text{]},$
	$0 \text{ ? } 0 \text{ [} 3 \text{ >= } Y, 6 \text{ != } 6 \text{]}\},$
$[(5 = 1) \wedge True] \vee False$	$\{(1 \text{ \& } 1) \text{ ? } 0 \text{ [} 5 = 1 \text{]},$
	$(0 \text{ \& } 1) \text{ ? } 0 \text{ [} 5 \text{ != } 1 \text{]}\}$

Usage of these and other similiar Σ -terms in atoms is described in the following transcription. On the left side are some Σ -facts, on the right side is their transcription into the Clingo language.

$p(True).$	$p(1).$
$p(False).$	$p(0).$
$p([(5 = 1) \wedge True] \vee False).$	$p((1 \text{ \& } 1) \text{ ? } 0) :- 5 = 1.$
	$p((0 \text{ \& } 1) \text{ ? } 0) :- 5 \text{ != } 1.$
$q(0, True).$	$q(0, 1).$
$q(123, (42 = 42)).$	$q(123, 1) :- 42 = 42.$
	$q(123, 0) :- 42 \text{ != } 42.$

1. In fact there are External functions which use Python or Lua API to compute values on grounding. You can read about them in [13]. I will not be showing them here as I do not use them in my implementation.

When transcribing rules of Example B.1.2, we have made some facts into rules by introducing new comparison predicates into the rule. In the section about grounding, we will however see, that these predicates will be removed during the grounding process.

B.1.2 Extensions of ASP in Clingo language

Unless specified otherwise, source of information in this section is [13].

Clingo language allows for many extensions of the Answer set programming. In this section, I will describe extensions, which will be used in the implementation part of this thesis.

Intervals and Pooling

When defining rules, it is often desired to define rules on a range of values. For this type of rule, Clingo language allows for the use of intervals and pools. Intervals and pools can be used in place of terms in both heads and bodies of rules.

For two whole numbers i, j , an interval $i..j$ means, that during grounding a rule containing this interval expands to multiple rules, one for each k s.t. $i \leq k \leq j$ in its place. If there are multiple such intervals, they expand one by one, latter ones expanding in all rules already partially expanded by previous intervals.

Example B.1.3. Consider the following program in Clingo language [13]:

```
grid(1..3, 1..3).
```

At the start of the grounding process, this program expands into the program:

```
grid(1, 1). grid(1, 2). grid(1, 3).
grid(2, 1). grid(2, 2). grid(2, 3).
grid(3, 1). grid(3, 2). grid(3, 3).
```

A pool is an expression in the form $(t_1; t_2; \dots; t_n)$. Similarly to the intervals during the grounding of a rule containing this pool, the rule expands to all rules containing $t_1, t_2, \dots, t_n$ in place of this pool. If there are multiple pools they expand consecutively.

Example B.1.4. Consider the following program in Clingo language:

```
grid((a; 3; #inf), (1; 2)).
```

At the start of the grounding process, this program expands into the program:

```
grid(a, 1).      grid(a, 2).
grid(3, 1).      grid(3, 2).
grid(#inf, 1).   grid(#inf, 2).
```

Example B.1.5. Consider the following program in Clingo language:

```
stairs(A, 1..A) :- A = 1..3.
```

When grounding this program, the grounder has to first expand the interval in the body. The head interval contains a variable A with value not yet known. After the first expansion, the following program emerges.

```
stairs(A, 1..A) :- A = 1.
stairs(A, 1..A) :- A = 2.
stairs(A, 1..A) :- A = 3.
```

Now there is only a single possible value of A in each expanded rule. A can be unified with this value and the head interval can be expanded. For better readability, I have removed comparison predicates from the body.

```
stairs(1, 1).
stairs(2, 1). stairs(2, 2).
stairs(3, 1). stairs(3, 2). stairs(3, 3).
```

Conditional literals

A conditional literal is an expression of the form:

$$l_0 : l_1, \dots, l_n$$

where each of $l_0, l_1, \dots, l_n$ are literals. This expression has a meaning of an inner implication. The conditional literal is consistent with a signature if and only if its head l_0 is consistent with this signature or its body $l_1, \dots, l_n$ is not consistent with this signature. It can be seen as an expression *put l_0 in its place if all literals $l_1, \dots, l_n$ are consistent with the partial solution.*

In the notation of conditional literals, use of commas , is compulsory. This is different from the rules, where commas and semicolons

can be interchanged freely. When the conditional literal is not the last literal of a rule body, a semicolon must be used as an end of this conditional literal. Conditional literals can be quite nontrivial, especially when used in head of a rule.

Example B.1.6. Consider following rules in Clingo language:

```
a :- b : c ; d.
a :- b : c , d.
```

The first rule has a body composed of a conditional literal `b : c` and a literal `d`. For this rule an equivalent program without conditional literals would be following:

```
x :- b.
x :- not c.
a :- x , d.
```

In this logic program the conditional literal was replaced with a fresh literal `x`. This literal is true if either the head of conditional literal is consistent or the body is not consistent.

The only difference between first and second rule is the comma or semicolon between literals `c` and `d`. Second rule has a body consisting only of conditional literal `b : c , d`. An equivalent program to this rule would be following:

```
x :- b.
x :- not c.
x :- not d.
a :- x.
```

Example B.1.7. Consider following rule in Clingo language:

```
a : b.
```

The rule consists of a single conditional literal in the head. If the literal `b` is not consistent with the partial solution, this rule acts as a rule with both head and body empty, thus invalidating any such solution. Rather than simply rewriting as a rule with `a` and `b` in the head and body respectively, the rule is equivalent to following program:

```
:- not b.
a :- b.
```

Aggregates

Aggregates are expressions of form

$$s_1 <_1 \alpha \{ t_1:L_1; \dots; t_n:L_n \} <_2 s_2,$$

where s_1, s_2 are terms, $<_1, <_2$ are comparison predicate symbols, α is an aggregate symbol, $t_1, \dots, t_n$ are terms and $L_1, \dots, L_n$ are sets of literals.

The full expression can be split into multiple parts. First, there is set of (conditional) terms²

$$T \equiv \{ t_1:L_1; \dots; t_n:L_n \}.$$

This set can be computed as a union of subsets

$$T \equiv \{t_1 : L_1\} \cup \dots \cup \{t_n : L_n\}.$$

If any of these subsets contains variables, it can be grounded similarly to rule. Grounded expressions can then be evaluated into sets of terms. The full expression is thus evaluated into set of terms containing all terms whose sets of literals are consistent with the partial solution.

Set T is evaluated in a set-like manner meaning if multiple its terms are the same, only a single one of them is included in the final set. When multiple occurrences of the same term is needed, special form of terms can be used. This form is tuple-like, only without outer brackets.

Should some term t_i be included in every such set regardless of consistency of any atoms (that is $L_i = \emptyset$), both the colon and part with literals $:L_i$ may be omitted.

Example B.1.8. Let S be a partial solution s.t. literals $p(1)$, $p(4)$, $p(5)$ and $p(7)$ are consistent with S .

The expression

$$T \equiv \{ 1; 3; 4; a; x \}$$

does not contain any variables nor literals, thus it is evaluated onto itself.

The expression

2. To distinguish between a comparison predicate symbol of Clingo language and a equality in the normal means, in this subsection I will use $\equiv$ rather than $=$ for the second.

$$T \equiv \{ 1:p(1); 3:p(3); 4:p(4), p(7) \}$$

does contain literals. When evaluated, it is first looked at the literals if they are consistent with the partial solution. For the first conditional term, $p(1)$ is consistent with S , thus 1 is included in the set. The second conditional term depends on literal $p(3)$ which is not consistent with S , 3 is not included in the set. Finally, the last conditional term depends on literals $p(4)$ and $p(7)$. Both of these literals are consistent with S , 4 is included in the set.

$$T \hookrightarrow \{ 1; 4 \}$$

The expression

$$T \equiv \{ Y-X:p(X), p(Y), X \leq Y \}$$

does contain both literals and variables. First we can start by grounding the expression. We are looking for pairs of literals $p(X)$, $p(Y)$ that comply to the comparison predicate $X \leq Y$.

$$T \hookrightarrow \{ 0:p(1), p(1); 3:p(1), p(4); 4:p(1), p(5); 6:p(1), p(7); 0:p(4), p(4); 1:p(4), p(5); 3:p(4), p(7); 0:p(5), p(5); 2:p(5), p(7); 0:p(7), p(7) \}$$

Now as we only took tuples of literals satisfying both the rules from the expression and being consistent with the partial solution, to evaluate T we can directly take terms in the set.

$$T \hookrightarrow \{ 0; 1; 2; 3; 4; 6 \}$$

In the evaluation of the last expression, we have lost the multiplicity of each term as it was evaluated as a set. To counteract that, we can add the lower parameter of each pair of literals to the terms.

$$T \equiv \{ Y-X, X:p(X), p(Y), X \leq Y \}$$

$$T \hookrightarrow \{ 0,1; 3,1; 4,1; 6,1; 0,4; 1,4; 3,4; 0,5; 2,5; 0,7 \}.$$

An aggregate acts as a function over set of conditional terms, aggregate symbol α specifies the type of function. Currently allowed aggregate symbols are $\#count$, $\#sum$, $\#sum+$, $\#min$ and $\#max$. These stand for count of different terms, sum over terms, sum over positive terms, minimal term and maximal term respectively. Max and min

aggregates do follow the same ordering as comparison predicates. If a term is composed of multiple parts, `#sum` and `#sum+` do their summation over the first part of term. If this part is not integer, the whole term is skipped. Summation over tuples or functions is not implemented. If the aggregate symbol α is omitted, it defaults to `#count`.

Aggregate returns a value in the form of a term. This value can then be compared with another term or assigned to variable. The same comparison predicate symbols can be used in place of $<_1, <_2$ as in comparison predicates. Both $s_1 <_1$ and $<_2 s_2$ can be omitted. If only $<_1$ or $<_2$ is omitted, comparison predicates defaults to $<=$.

Example B.1.9. In this example we will work with expressions from Example B.1.8. Let S be a partial solution s.t. literals $p(1), p(4), p(5)$ and $p(7)$ are consistent with S .

The rule

$$a(X) :- X = \#count\{ 1; 3; 4; a; x \}.$$

adds a single atom $q(5)$ into the solution as all elements of the aggregate set are terms. Here only a single left-hand comparison predicate is used. The right-hand predicate is omitted.

The rule

$$a :- 2 \#sum\{ 1:p(1); 3:p(3); 4:p(4), p(7) \} <= 7$$

adds a single atom q into the solution. As already shown, the inner expression evaluates into

$$\{ 1; 4 \},$$

sum of this set is equal to 5, which satisfies comparison $2 <= 5 <= 7$. The left-hand side comparison predicate symbol is omitted, thus by default is $<=$.

Both rules

$$a(X) :- X = \#max\{ Y-X:p(X), p(Y), X <= Y \}.$$

and

$$a(X) :- X = \#max\{ Y-X, X:p(X), p(Y), X <= Y \}.$$

add a single atom $a(6)$ into the solution. If tuple without brackets is used in the definition of term, the $\#max$ aggregate does take only the first element into account.

If both terms are needed, one can use the following rule:

$$a(X, Y) :- (X, Y) = \#max\{ (Y-X, X) : p(X), p(Y), X \leq Y \}.$$

This time each term of the aggregate set is a single tuple. As written in Basic syntax, terms are compared in lexicographic manner.

A special form of aggregates is also allowed in the head of a rule. This type of aggregates, called head aggregates, are of form:

$$s_1 <_1 \alpha \{ t_1:l_1:L_1; \dots; t_n:l_n:L_n \} <_2 s_2,$$

where s_1, s_2 are terms, $<_1, <_2$ are comparison predicate symbols, α is an aggregate symbol, $t_1, \dots, t_n$ are terms, $l_1, \dots, l_n$ are literals and $L_1, \dots, L_n$ are sets of literals.

When evaluating the set, term t_i is included in the set if both literal l_i and all literals from L_i are consistent with the solution. Additionally, if the aggregate forms a head of rule r and body of r is consistent with the solution, the aggregate can introduce any number of literals l_i for such all literals L_i are consistent with the solution. This is especially useful for generating an input space of logic program.

For all $s_1, s_2, <_1, <_2, \alpha, L_i$ the default values and rules for omitting are still the same as when using the aggregates in the body. Specially, the expression

$$s_1 <_1 \{ l_1:L_1; \dots; l_n:L_n \} <_2 s_2,$$

is equivalent to

$$s_1 <_1 \#count\{ t_1:l_1:L_1; \dots; t_n:l_n:L_n \} <_2 s_2,$$

where all t_i are distinct.

Example B.1.10. Let the logic program consist of a single fact

$$\{a; b; c; d; e\}.$$

This fact is a syntactic shortcut of $fact^3$

3. The following expression is not a valid rule in clingo language as it does not contain $\emptyset$. I choose to include it into the notation here to improve readability.

```
#count{ 1:a:⊘; 2:b:⊘; 3:c:⊘; 4:d:⊘; 5:e:⊘ }.
```

As an empty set of literals is consistent with any solution, any of atoms a, b, c, d, e can be included into the solution by this rule. As there is no constraint on the result of this aggregate, any subset of this set of atoms can be included into the solution.

The statement in the Example B.1.10 allows us to easily define the input space of logic program.

Show statements

Often only a part of a solution is needed, while most of the solution can be hidden. For this, Clingo does support show statement. Such statement takes one of following forms:

```
#show p/n.
#show t:L1, ..., Ln.
#show.
```

In the first form, atoms with predicate symbol p with exactly n parameters are shown on the output. If at least one such atom is included in the solution, only atoms specified by `#show` statements are shown. A predicate symbol and its classical negation is taken as two distinct symbols.

The second form allows for showing terms if some literals are consistent with the solution. For a term t and literals $L_1, \dots, L_n$, term t is shown on the output if all literals $L_1, \dots, L_n$ are consistent with the solution. This form does not hide any atoms from the solution.

Finally, the third form specifies, that no atom other than these specified by show statements in the first form should be shown on the output.

B.2 Gringo

Gringo [25] is software that grounds logic program in Clingo language and translates it into the Aspif format that is readable by logic program solver Clasp. Gringo first resolves every rule with variables into (possibly multiple) variable-free rules and then changes format

of the program into Clasp-readable Aspif. Gringo thus can introduce new atoms that were not obvious from the Clingo program. Full specification of Aspif language is written in [27].

B.2.1 Aspif format

Aspif (ASP intermediate format) language consists of statements, each on its own line. First line of file is a header of form

$$\text{asp } v_m \ v_n \ v_r \ t_1 \ \dots \ t_k$$

where v_m, v_n and v_r are versions of major, minor and revision numbers respectively and each t_i is a tag. Then follow lines with rules and statements translated from program. Last line of Aspif format file is a single 0.

For the translation from Clingo to Aspif language, Gringo introduces mapping M of literals onto positive numbers. Each literal is assigned a positive number, which identifies it in the whole Aspif program. To assert the constraint that no solution contains both an atom and its classical negation, for each such tuple of literals $\varphi, \neg\varphi$, Gringo introduces new rule to forbid such solutions:

$$:- \varphi, \neg\varphi.$$

In this thesis only rules and show statements are relevant.

B.2.2 Rule statements

Rule in Aspif has form of

$$1 \ H \ B$$

in which head H has form of

$$h \ m \ a_1 \ \dots \ a_m$$

where $h \in \{0, 1\}$, $m \geq 0$, $\forall i \in \{1, \dots, m\}. a_i \in \mathbb{N}^+$. Parameter h determines whether head of this rule is disjunction (0) or choice (1), m determines number of literals and a_i are literals mapped to positive integers.

Body B is called normal if it has form of

$$0 \ n \ l_1 \ \dots \ l_n$$

where n is the number of literals in statement and for each literal φ corresponding to l_i , $l_i = M(\varphi)$ if φ is positive in the sense of the default negation, otherwise $l_i = -M(\varphi)$. Literals of the normal body are in conjunction meaning all its positive literals and none of its negative literals have to be consistent with the answer set in order for the head of rule to be used.

The other type of body B is called weight body. Its form is

$$1 \ b \ n \ l_1 \ w_1 \ \dots \ l_n \ w_n$$

Parameter $b \geq 0$ determines lower bound, n the length of rule body, for each literal φ corresponding to l_i , $l_i = M(\varphi)$ if φ is positive in the sense of the default negation, otherwise $l_i = -M(\varphi)$ and $w_i \geq 1$ is the weight assigned to this literal. In weight body, the head of the rule is used if the sum of weights of expressions consistent with the answer set is greater or equal to the lower bound.

Example B.2.1. Consider the following program in Clingo language:

```

1 p(a).
2 p(b) :- p(a).
3 p(c) :- p(a), p(b).

5 q(1..3).

7 r(1) :- not r(2).
8 r(2) :- not r(1).

10 s(true) :- q(1) : q(2), q(3).

12 t(A) :- A = #count { X: q(X) }.
```

When grounding the program, first a set of ground rules will be generated. In this case, non ground rules are on lines 5 and 12. Expansion of the rule on line 5 is going to be easy, it is going to expand into 3 facts, with values 1, 2 or 3 as the parameter.

```
5 q(1). q(2). q(3).
```

The rule on line number 12 will be expanded in two steps. First, the set of conditional terms will be expanded.

```
12 t(A) :- A = #count{ 1: q(1); 2: q(2); 3: q(3) }.
```

Now that both the minimal and maximal number of terms are known, the variable A can be grounded.

```

12 t(0) :- 0 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
13 t(1) :- 1 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
14 t(2) :- 2 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
15 t(3) :- 3 = #count{ 1: q(1); 2: q(2); 3: q(3) }.

```

After the grounding process, the full program will have the form of:

```

1 p(a).
2 p(b) :- p(a).
3 p(c) :- p(a), p(b).

5 q(1). q(2). q(3).

7 r(1) :- not r(2).
8 r(2) :- not r(1).

10 s(true) :- q(1) : q(2), q(3).

12 t(0) :- 0 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
13 t(1) :- 1 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
14 t(2) :- 2 = #count{ 1: q(1); 2: q(2); 3: q(3) }.
15 t(3) :- 3 = #count{ 1: q(1); 2: q(2); 3: q(3) }.

```

Finally, Gringo will process these grounded rules into statements⁴. On the first line of the resulting Aspf file informations about the version will be written.

```
1 1 0 0
```

Then, the fact and rules on lines 1–3 will be translated. Literals p(a), p(b) and p(c) will be mapped to numbers 1, 2 and 3.

```

2 1 0 1 1 0 0
3 1 0 1 2 0 1 1
4 1 0 1 3 0 2 1 2

```

The first statement has a disjunctive head containing a single literal with assigned number 1, that is the literal p(a). It also has normal body

4. In fact, Gringo will do some optimizations already in the time of translation into statements. For instance, it does see that literal p(a) has to always be included in the answer set, thus also the body of rule on line 2 does always hold and line 2 would be substituted for a fact. The mapping of literals to numbers is also not the same as Gringo would use.

containing no literal. As it does not contain any literal, the statement is a fact.

The second statement also has a disjunctive head containing a single literal $p(b)$. Its body is normal, containing a single literal $p(a)$, thus this rule ensures literal $p(b)$ if $p(a)$ is consistent.

The third statement also has a disjunctive head containing a single literal $p(c)$. Its body is normal, containing 2 literals $p(a)$ and $p(b)$.

Line 5 contains only three facts. Each of them is going to be translated into a single rule, similar to the fact from line 1. Literals $q(1)$, $q(2)$ and $q(3)$ will be assigned numbers 4, 5, 6.

```
5 1 0 1 4 0 0
6 1 0 1 5 0 0
7 1 0 1 6 0 0
```

Rules on lines 7–8 contain default negation. According to the definition, negated values of numbers assigned to corresponding literals will be used in statement bodies. Literals $r(1)$ and $r(2)$ will be assigned numbers 7 and 8.

```
8 1 0 1 7 0 1 -8
9 1 0 1 8 0 1 -7
```

The rule on line 10 does use conditional literal. To translate this into statements, Gringo uses trick similar to the one used in the Example B.1.6. For each conditional literal $H : B$ it first creates a rule with a new literal φ in the head and the body B as the body of the rule. Then it adds another literal ψ that corresponds to the value of the conditional literal and another two rules asserting that the literal ψ is consistent with solution if H is consistent or φ is not consistent with the solution.

The rule expands into the following rules in the Clingo language:

```
 $\varphi$  :-  $q(2)$ ,  $q(3)$ .
 $\psi$  :-  $q(1)$ .
 $\psi$  :- not  $\varphi$ .
s(true) :-  $\psi$ .
```

These rules are then directly translated into Aspif statements. Literals $q(1)$, $q(2)$ and $q(3)$ are already assigned numbers 4, 5 and 6. Literals φ , ψ and $s(\text{true})$ are assigned numbers 9, 10 and 11.

```
10 1 0 1 9 0 2 5 6
11 1 0 1 10 0 1 4
```

```

12 1 0 1 10 0 1 -9
13 1 0 1 11 0 1 10

```

The last set of rules do contain aggregates. These four rules only differ in the number of literals being true in the solution. To be transcribe these rules into statements in which only the lower bound is defined (where only the operator $\leq$ is allowed), Gringo splits each equality into two inequalities. Any expression in form of (A and B are head and rest of body):

$A :- K = \alpha\{\dots\}, B.$

is first rewritten as three rules using only operator $\leq$ ⁵:

```

 $\varphi :- K \leq \alpha\{\dots\}, B.$ 
 $\psi :- K + 1 \leq \alpha\{\dots\}, B.$ 
 $A :- \varphi, \text{ not } \psi.$ 

```

where φ and ψ are fresh literals. These expressions are equivalent as the value of an aggregate is always a whole number. The default negation in the last rule negates the $\leq$ operator of the second rule making its meaning close to $>$.

In our example, there are 4 such rules, thus we will be using 8 fresh literals. $t(0)$, $t(1)$, $t(2)$ and $t(3)$ are assigned numbers 12, 13, 14 and 15. Fresh literals are assigned numbers 16, 17, 18, 19, 20, 21, 22, 23. $q(1)$, $q(2)$ and $q(3)$ are already assigned numbers 4, 5 and 6.

```

14 1 0 1 16 1 0 3 4 1 5 1 6 1
15 1 0 1 17 1 1 3 4 1 5 1 6 1
16 1 0 1 12 0 2 16 -17
17 1 0 1 18 1 1 3 4 1 5 1 6 1
18 1 0 1 19 2 1 3 4 1 5 1 6 1
19 1 0 1 13 0 2 18 -19
20 1 0 1 20 2 1 3 4 1 5 1 6 1
21 1 0 1 21 3 1 3 4 1 5 1 6 1
22 1 0 1 14 0 2 20 -21
23 1 0 1 22 3 1 3 4 1 5 1 6 1
24 1 0 1 23 4 1 3 4 1 5 1 6 1
25 1 0 1 15 0 2 22 -23

```

5. In fact, Gringo treats the lowest and highest values differently. As there is no way to undershoot or overshoot these values, it does not create the first or second rule respectively in these cases.

The last line of the Aspif program will be a single zero. Full program⁶ will take form of:

```

1 1 0 0
2 1 0 1 1 0 0
3 1 0 1 2 0 1 1
4 1 0 1 3 0 2 1 2
5 1 0 1 4 0 0
6 1 0 1 5 0 0
7 1 0 1 6 0 0
8 1 0 1 7 0 1 -8
9 1 0 1 8 0 1 -7
10 1 0 1 9 0 2 5 6
11 1 0 1 10 0 1 4
12 1 0 1 10 0 1 -9
13 1 0 1 11 0 1 10
14 1 0 1 16 1 0 3 4 1 5 1 6 1
15 1 0 1 17 1 1 3 4 1 5 1 6 1
16 1 0 1 12 0 2 16 -17
17 1 0 1 18 1 1 3 4 1 5 1 6 1
18 1 0 1 19 2 1 3 4 1 5 1 6 1
19 1 0 1 13 0 2 18 -19
20 1 0 1 20 2 1 3 4 1 5 1 6 1
21 1 0 1 21 3 1 3 4 1 5 1 6 1
22 1 0 1 14 0 2 20 -21
23 1 0 1 22 3 1 3 4 1 5 1 6 1
24 1 0 1 23 4 1 3 4 1 5 1 6 1
25 1 0 1 15 0 2 22 -23
26 0

```

It is very important to choose the right representation of a problem in answer set programming. As shown in the example above, some representations may lead to very large size of a grounded logic program. When using variables in the head of a rule, one has to be aware of the number of literals and rules added through the grounding process.

B.2.3 Show statements

Show statement is for specification of output, they result from #show directive. Each show statement is of form:

6. As there are no show statements in the former logic program, the Aspif program would contain a show statement for each literal of the grounded program.

$$4 \ m \ s \ n \ l_1 \ \dots \ l_n$$

where m is length of string s , s is string with name, n is number length of condition and l_i are literals. The show statement prints the string s if all literals l_i are consistent with the answer set.

Example B.2.2. In the logic program from Example B.2.1 in Clingo format, there are no show statements. Thus for every literal l with of the grounded logic program mapped to number $M(l)$, a single show statement of form:

$$4 \ ||L|| \ L \ 1 \ M(l)$$

would be added into the logic program in Aspif format.

To get the Aspif representation mentioned in this Example, a single show statement `#show.` could be used.

If show statement `#show q/1.` was added to the former logic program, the following set of show statements would be included in its Aspif representation:

```
26 4 4 q(1) 1 4
27 4 4 q(2) 1 5
28 4 4 q(3) 1 6
```

B.3 Clasp

Clasp is the solver of the Clingo framework. It takes logic program in the Aspif format to search for answer sets of a grounded logic program. The solver approaches the inference using the unit propagation of nogoods [28].

For the inference of answer sets of some logic program, Clasp uses backpropagation over a tree of partial solutions. Each time the it sees a partial solution that is not an answer set of the problem, it derives new constraints on the solution which it propagates up the tree of partial solutions. This allows Clasp to prune large branches of partial solutions.

For model (answer sets) counting, Clasp uses a method called model enumeration. To determine that a set is an answer set, it needs to evaluate the exact set. This means, that the time complexity of an enumeration is always dependent on the number of answer sets.

C Code of BNN verifier

The full code of the verifier implemented in this thesis together with the examples can be found either in the Thesis archive in the IS MU or in the Github repository https://github.com/Ardnij123/BNN_verification

D Specifications of system used for evaluation

For the evaluation a machine with freshly installed Arch linux distribution was used. The used machine did not have any window manager nor desktop environment setup. No applications other than the system ran on the machine at the time of evaluation. You can find both hardware and software specifications of the machine in tables below.

D.1 Hardware

Model	Dell Inc. G3 3579
CPU	8 x Intel Core i5-8300H, 2.3 GHz
RAM	15867 MiB

D.2 Software

Distribution	Arch Linux x86_64
Kernel	Linux 6.12.4-arch1-1
Clingo	5.7.1
Clasp	3.3.10
Python	3.12.7
NumPy	2.2.0

E Evaluation of models

Distance	$d = 0$		$d = 1$		$d = 2$		$d = 3$		$d = 4$	
M1	0.864	0.00%	1.539	7.92%	11.240	8.57%	198.248	31.03%	—	—
M2	0.234	0.00%	0.365	0.00%	3.642	0.13%	115.881	3.69%	—	—
M3	1.147	0.00%	4.075	0.00%	—	—	—	—	—	—
M4	0.017	0.00%	0.024	0.00%	0.083	0.00%	1.070	8.40%	11.463	17.34%
M5	1.511	0.00%	11.936	0.00%	—	—	—	—	—	—
M6	0.428	0.00%	7.093	5.94%	18.967	9.95%	221.228	25.35%	—	—
M7	0.094	0.00%	0.455	0.00%	2.415	3.84%	50.337	2.90%	—	—
M8	0.043	0.00%	0.064	64.70%	0.086	70.80%	0.102	80.91%	0.159	84.74%
M9	0.018	0.00%	0.024	0.00%	0.048	3.14%	0.135	24.22%	0.751	28.87%
M10	0.137	0.00%	0.526	70.58%	0.470	77.37%	0.579	86.65%	0.943	89.75%
M11	0.052	0.00%	0.182	46.15%	0.150	63.49%	0.343	69.04%	1.309	77.44%
M12	0.961	0.00%	29.968	0.00%	—	—	—	—	—	—

Table E.1: Computation time for verification of all models on $HD(I0, d)$ for $d \in \{0, 1, 2, 3, 4\}$. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Symbols — in the table mean that less than 3 of 4 evaluations finished before timeout. Timeout was set to 600 s.

Fixed bits	$F = 0$		$F = 4$		$F = 8$		$F = 12$		$F = 16$		$F = 20$		$F = 24$	
M1	0.822	0.00%	0.892	18.75%	0.937	64.45%	1.267	79.22%	7.608	70.97%	121.686	87.18%	—	—
M2	0.222	0.00%	0.229	0.00%	0.243	40.23%	0.345	39.45%	2.004	41.33%	43.101	70.51%	—	—
M3	1.153	0.00%	1.178	0.00%	1.215	0.00%	1.628	0.00%	3.145	0.00%	27.866	0.00%	—	—
M4	0.016	0.00%	0.018	12.50%	0.019	77.34%	0.029	86.18%	0.192	61.21%	4.087	70.44%	78.504	74.61%
M5	1.448	0.00%	1.470	0.00%	1.500	0.00%	1.742	0.00%	2.675	0.00%	18.440	0.00%	127.466	0.00%
M6	0.410	0.00%	0.700	12.50%	1.102	55.46%	2.571	75.31%	20.863	78.92%	320.730	78.24%	—	—
M7	0.892	0.00%	0.094	0.00%	0.129	29.68%	0.235	69.60%	1.574	64.01%	26.100	66.33%	447.620	73.23%
M8	0.043	0.00%	0.059	93.75%	0.088	89.45%	0.162	83.17%	1.860	95.64%	—	—	—	—
M9	0.017	0.00%	0.018	31.25%	0.020	77.73%	0.031	86.01%	0.278	79.37%	4.342	64.81%	100.005	84.28%
M10	0.130	0.00%	0.449	75.00%	0.452	86.71%	1.066	94.94%	9.596	98.04%	—	—	—	—
M11	0.051	0.00%	0.118	93.75%	0.116	88.28%	0.301	84.69%	2.960	91.77%	61.241	90.74%	—	—
M12	0.912	0.00%	0.921	0.00%	1.212	0.00%	1.904	0.00%	2.437	0.00%	5.687	0.00%	11.680	0.00%

Table E.2: Computation time for verification of all models on $R(I0, \{F + 1, \dots, n\})$ where n is size of input of model and $F \in \{0, 4, 8, 12, 16, 20, 24\}$. Values in the table were obtained using methodology described in Section 4.1.1 (avg. of 3 best of 4). Symbols — in the table mean that less than 3 of 4 evaluations finished before timeout. Timeout was set to 600 s.

Bibliography

1. ZHANG, Yedi; ZHAO, Zhe; CHEN, Guangke; SONG, Fu; CHEN, Taolue. BDD4BNN: a BDD-based quantitative analysis framework for binarized neural networks. In: *International Conference on Computer Aided Verification*. Springer, 2021, pp. 175–200.
2. GEBSER, Martin; KAMINSKI, Roland; KAUFMANN, Benjamin; SCHAUB, Torsten. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*. 2019, vol. 19, no. 1, pp. 27–82.
3. WEI, Jason; BOSMA, Maarten; ZHAO, Vincent Y.; GUU, Kelvin; YU, Adams Wei; LESTER, Brian; DU, Nan; DAI, Andrew M.; LE, Quoc V. Finetuned Language Models Are Zero-Shot Learners. *arXiv e-prints*. 2021, arXiv:2109.01652. Available from doi: 10.48550/arXiv.2109.01652.
4. JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. 2017. Available from arXiv: 1712.05877 [cs.LG].
5. HAN, Song; MAO, Huizi; DALLY, William J. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. 2016. Available from arXiv: 1510.00149 [cs.CV].
6. HUBARA, Itay; COURBARIAUX, Matthieu; SOUDRY, Daniel; EL-YANIV, Ran; BENGIO, Yoshua. Binarized Neural Networks. In: LEE, D.; SUGIYAMA, M.; LUXBURG, U.; GUYON, I.; GARNETT, R. (eds.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2016, vol. 29. Available also from: https://proceedings.neurips.cc/paper_files/paper/2016/file/d8330f857a17c53d217014ee776bfd50-Paper.pdf.
7. AGRAWAL, Tanay. *Binarized Neural Network (BNN) and its implementation in machine learning* [online]. 2023-08. [visited on 2024-04-11]. Available from: <https://neptune.ai/blog/binarized-neural-network-bnn-and-its-implementation-in-ml>.

8. CHENG, Chih-Hong; NÜHRENBURG, Georg; HUANG, Chung-Hao; RUESS, Harald. Verification of Binarized Neural Networks via Inter-neuron Factoring. In: PISKAC, Ruzica; RÜMMER, Philipp (eds.). *Verified Software. Theories, Tools, and Experiments*. Cham: Springer International Publishing, 2018, pp. 279–290. ISBN 978-3-030-03592-1.
9. ZHANG, Yedi; ZHAO, Zhe; CHEN, Guangke; SONG, Fu; CHEN, Taolue. Precise Quantitative Analysis of Binarized Neural Networks: A BDD-based Approach. *ACM Trans. Softw. Eng. Methodol.* 2023, vol. 32, no. 3. ISSN 1049-331X. Available from doi: 10.1145/3563212.
10. LIFSCHITZ, V. What is answer set programming? In: *Proc. 23rd AAAI Conf. on Artificial Intelligence, 2008*. 2008, pp. 1594–1597.
11. ANGER, Christian; KONCZAK, Kathrin; LINKE, Thomas; SCHAUB, Torsten. A Glimpse of Answer Set Programming. *Künstliche Intell.* 2005, vol. 19, no. 1, p. 12.
12. GELFOND, Michael; LIFSCHITZ, Vladimir. Classical Negation in Logic Programs and Disjunctive Databases. *New Generation Computing*. 1991, vol. 9, pp. 365–385. Available from doi: 10.1007/BF03037169.
13. GEBSER, Martin; KAMINSKI, Roland; KAUFMANN, Benjamin; LINDAUER, Marius; OSTROWSKI, Max; ROMERO, Javier; SCHAUB, Torsten; THIELE, S; WANKO, P. *Potassco guide version 2.2.0*. 2019.
14. BALUTA, Teodora; SHEN, Shiqi; SHINDE, Shweta; MEEL, Kuldeep S.; SAXENA, Prateek. *Quantitative Verification of Neural Networks And its Security Applications*. 2019. Available from arXiv: 1906.10395 [cs.CR].
15. LECUN, Yann. *THE MNIST DATABASE of handwritten digits* [online]. [visited on 2024-04-11]. Available from: <http://yann.lecun.com/exdb/mnist/>.
16. HUBARA, Itay; COURBARIAUX, Matthieu; SOUDRY, Daniel; EL-YANIV, Ran; BENGIO, Yoshua. Binarized Neural Networks. *ArXiv*. 2016, vol. abs/1602.02505.

17. BISHOP, Christopher M. Neural networks for pattern recognition. In: 1995. Available also from: <https://api.semanticscholar.org/CorpusID:60563397>.
18. COURBARIAUX, Matthieu; HUBARA, Itay; SOUDRY, Daniel; EL-YANIV, Ran; BENGIO, Yoshua. *Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1*. 2016. Available from arXiv: 1602.02830.
19. HARMELEN, Frank; LIFSCHITZ, Vladimir; PORTER, Bruce. The Handbook of Knowledge Representation. *Elsevier Science San Diego, USA*. 2007, p. 1034.
20. DELGRANDE, Jim. *Answer Set Programming* [online]. [N.d.]. [visited on 2024-04-27]. Available from: <https://www2.cs.sfu.ca/CourseCentral/721/jim/ASP1.Intro.pdf>.
21. LIFSCHITZ, Vladimir. Twelve Definitions of a Stable Model. In: GARCIA DE LA BANDA, Maria; PONTELLI, Enrico (eds.). *Logic Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 37–51. ISBN 978-3-540-89982-2.
22. ZARBA, Calogero G. *Many-Sorted Logic* [online]. 2006. [visited on 2024-10-26]. Available from: <https://web.archive.org/web/20070929131504/http://react.cs.uni-sb.de/~zarba/snow/ch01.pdf>.
23. EITER, Thomas; GOTTLOB, Georg; MANNILA, Heikki. Disjunctive datalog. *ACM Trans. Database Syst.* 1997, vol. 22, no. 3, pp. 364–418. ISSN 0362-5915. Available from DOI: 10.1145/261124.261126.
24. SCHLIPF, J.S. The Expressive Powers of the Logic Programming Semantics. *Journal of Computer and System Sciences*. 1995, vol. 51, no. 1, pp. 64–86. ISSN 0022-0000. Available from DOI: <https://doi.org/10.1006/jcss.1995.1053>.
25. GEBSER, Martin; KAMINSKI, Roland; KAUFMANN, Benjamin; SCHAUB, Torsten. Multi-shot ASP solving with clingo. *CoRR*. 2017, vol. abs/1705.09811.

26. GEBSER, MARTIN; HARRISON, AMELIA; KAMINSKI, ROLAND; LIFSCHITZ, VLADIMIR; SCHAUB, TORSTEN. Abstract gringo. *Theory and Practice of Logic Programming*. 2015, vol. 15, no. 4–5, pp. 449–463. ISSN 1475-3081. Available from doi: 10.1017/s1471068415000150.
27. GEBSER, Martin; KAMINSKI, Roland; KAUFMANN, Benjamin; OSTROWSKI, Max; SCHAUB, Torsten; WANKO, Philipp. *Theory Solving Made Easy with Clingo 5 (Extended Version)*. 2016.
28. GEBSER, Martin; KAUFMANN, Benjamin; SCHAUB, Torsten. Conflict-driven answer set solving: From theory to practice. *Artif. Intell.* 2012, vol. 187, pp. 52–89.
29. HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der; GOMMERS, Ralf; VIRTANEN, Pauli; COURNAPEAU, David; WIESER, Eric; TAYLOR, Julian; BERG, Sebastian; SMITH, Nathaniel J.; KERN, Robert; PICUS, Matti; HOYER, Stephan; KERKWIJK, Marten H. van; BRETT, Matthew; HALDANE, Allan; RÍO, Jaime Fernández del; WIEBE, Mark; PETERSON, Pearu; GÉRARD-MARCHANT, Pierre; SHEPPARD, Kevin; REDDY, Tyler; WECKESSER, Warren; ABBASI, Hameer; GOHLKE, Christoph; OLIPHANT, Travis E. Array programming with NumPy. *Nature*. 2020, vol. 585, no. 7825, pp. 357–362. Available from doi: 10.1038/s41586-020-2649-2.
30. HONG, Don; WANG, Jianzhong; GARDNER, Robert. *Real Analysis with an Introduction to Wavelets and Applications*. Burlington: Academic Press, 2005. ISBN 978-0-12-354861-0. Available from doi: <https://doi.org/10.1016/B978-012354861-0/50003-8>.